Universitat Rovira i Virgili (URV) y Universitat Oberta de Catalunya (UOC)

Master in Computational and Mathematical Engineering

FINAL MASTER PROJECT

Area: YYY

xxx title of the work xxx

xxx subtitle (if it is the case) xxx

Autor: Complete name of the student

Tutor: Complete name of the director

Barcelona, April 27, 2026

Dr./Dra. (name) ....., certifies that the student (name) ..... has elaborated the work under his/her direction and he/she authorizes the presentation of this memory for its evaluation.

Director's signature:

Credits/Copyright

A page with the specification of credits / copyright for the project (either application on one hand and documentation on the other, or unified), as well as the use of brands, products or services of third parties (including source codes). If a person other than the author collaborated in the project, his identity must be made explicit and what he did.

The most usual case is exemplified below, although it can be modified by any other alternative:

FINAL PROJECT SHEET

Title:	Descriptive of the project
Autor:	Name and surname(s)
Tutor:	Name and surname(s)
Date (mm/yyyy):	Date of delivery
Program:	Master in Computational and Mathematical Engineering
Area:	Name of the FMP area
Language:	English
Key words	Maxim 3 key words

Dedicatory / Quote

Brief words of dedicatory or quote.

Acknowledgments

If deemed appropriate, mention the persons, companies or institutions that have contributed to the realization of this project.

Abstract

Text with the synthesis of the project, that is, a text in which the definition of the project / problem addressed, its objectives / methods of resolution, and the results and conclusions are briefly explained (it can not be a list, but a text continuous written in a structured way). If it is necessary to put a reference in this text, it will be annotated at the bottom of the same page. In this section you can use a more literary and colloquial language than for the rest of the document.

In case of not writing the the document in English, it will be necessary to write the Abstract in duplicate. One of the versions must be textbf obligatorily in English. The other version can be written in Catalan or Spanish, in the language used for the rest of the report. The word Abstract will be changed to " textbf Resum " or " textbf Resumen " in the Catalan and Spanish versions, respectively.

Recommended length: 250 words maximum.

How to write a good abstract (in English):

http://www.ece.cmu.edu/ koopman/essays/abstract.html

Keywords: Work keywords separated by commas. For example for this document they could be Model, Guideline, Template, Memory, Final Master Project / Master

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Planning

1.1 Introduction

This Final Master's Project focuses on designing and implementing a distributed C++ program for simulating flood events. Floods are sudden and violent events that have serious economic and human consequences. Specifically, this project was launched in response to the DANA (Depresión Aislada en Niveles Altos) that struck Valencia, Spain, on October 29, 2024. This was not just any cold snap, but the most catastrophic one in the country's recent history. For this reason, it has been used as a case study to test modeling capabilities.

The proposed system uses a Cellular Automata engine to model water flow. The terrain is divided into a grid of cells with topographic data from Geographic Information Systems. The simulator applies rules to calculate changes in water depth and velocity over time. This approach allows for processing of large-scale Digital Elevation Models while maintaining precision.

A key feature of this project is its distributed architecture. The system operates in an environment using the MQTT protocol for communication between the simulation engine and external visualization modules.

This structure is a step towards creating a Digital Twin for disaster management. By integrating real-time data feeds with the Cellular Automata model the system aims to provide a synchronized representation of the flood. The project aims to bridge the gap between flood modeling and practical emergency response.

By simulating the Valencia flood the work shows how distributed computing and cellular modeling can be used to identify high-risk zones and predict flood evolution in time. This provides a tool, for both analyzing past floods and mitigating future crises.

1.2 Motivation and Justification

The reason we are doing this research is because of the increasing number of weather events, which are a big problem for cities and emergency services. The big flood in Valencia on October 29 2024

showed us that our current systems for predicting these events are not good enough and we need tools to simulate them.

Scientific and Technical Necessity

The old way of modeling water flow uses math that takes a lot of computer power and is hard to use in real-time. This project uses something called Cellular Automata, which is a simpler and more efficient way to model how water moves. This is important because we need to be able to do this accurately.

Innovation through Distributed Architecture

There is a gap between the tools we use to simulate disasters and the tools we use to respond to them. This project uses a way of connecting different systems called MQTT to make it easier for different tools to work together. This means we can:

- Use tools to visualize the data in real-time like 3D models or maps.
- Make the system bigger or smaller depending on what we need and connect it to sensors in the field.
- Create a copy of a real place like a river basin, which can help us understand what might happen in an emergency.

Socio-Economic Impact

This project is not about the technology, it is also about how it can help people. By making a tool that can simulate the flood in Valencia we can:

1. Look back, at what happened. See what made the flood so bad.
2. Make a system that can help us make decisions in real-time, like where to evacuate people and how to use our resources.

1.3 Objectives

1.3.1 General Objective

The main goal is to create a system that can simulate phenomena such as floods. It will use Cellular Automata and the C++ programming language to make it work well. The project wants to provide a way to analyze disasters and monitor them in time. This system will be like a Digital Twin.

1.3.2 Specific Objectives

To achieve the goal we have set the following goals:

- **State-of-the-Art Analysis**: Perform a comprehensive review of existing scientific literature regarding flood simulation, focusing on Cellular Automata and C++ flow models suitable for water physics.
- **CA Engine Design**: Define the transition rules and neighborhood structures—such as Moore or Von Neumann—to accurately represent terrain cells and water depth dynamics.
- **GIS Integration**: Develop a specialized module to read and parse Geographic Information System (GIS) data, specifically Digital Elevation Models (DEM), to initialize the simulation environment based on real-world topography.
- **Distributed Communication**: Implement a communication layer based on the MQTT protocol to allow the simulator to interact with external visualizers and data sources, enabling its operation within a distributed ecosystem.
- **Model Validation**: Conduct exhaustive unit testing and validation by comparing simulation outputs against known data from the Valencia DANA event to ensure the reliability of the water propagation logic.
- **Visualization and Analysis**: Generate tools for processing output data to facilitate the interpretation of the impact on the terrain.

1.4 Project Planning and Management

The successful execution of the Final Master's Project (FMP), which focuses on developing a scientific C++ program for flood simulation, requires robust planning and diligent management. This course is weighted at 18 ECTS, equating to an estimated dedicated effort of approximately 450 hours. The timeline presented below serves as a guiding roadmap, designed to track progress against key project milestones and deliverables.

1.4.1 Work Methodology

The project methodology is structured around the four mandatory phases defined for the FMP, integrated with a phased software development approach. This hybrid methodology ensures that the core theoretical and design elements (Phases 1 & 2) are completed before critical implementation and validation (Phase 3) and final documentation (Phase 4). The iterative nature of the plan allows for

necessary adjustments and includes dedicated time for thorough testing and writing throughout the academic year.

1.4.2 Project Schedule: Gantt Chart

The following Gantt chart (Figure 1.1) illustrates the proposed 24-week timeline for the project, distributing the estimated 450 working hours across the defined phases. Time buffers have been implicitly integrated into the task durations to account for unforeseen issues, necessary code refactoring, and academic holidays.

1.4.3 Details of Activities

The following table provides a detailed breakdown of the tasks defined in the timeline, specifying the scope of work for each activity.

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

ID	Activity	FMP Phase	Description (2-5 Lines)
PHASE 1: PLANNING			
P1.1	State-of-the-Art Review	Phase 1	Review existing scientific literature on flood simulation, focusing on Cellular Automata (CA) and C++ flow models. Key CA models suitable for water physics will be identified and the core references for the thesis will be collected.
P1.2	Objectives and Scope Definition	Phase 1	Clearly specify the flood scenarios the program will simulate (e.g., 2D vs. 3D, terrain type). Success criteria will be established, and the exact GIS data format required for input topography will be defined.
P1.3	Development Environment Setup	Phase 1	Install and configure the C++ compiler (including necessary GIS libraries like GDAL), the version control system (Git), and development tools. A basic LaTeX project structure will be created to initiate the project documentation.
PHASE 2: PROBLEM DESCRIPTION AND PROPOSAL			

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

ID	Activity	FMP Phase	Description (2-5 Lines)
P2.1	Cellular Automata (CA) Design	Phase 2	Define the transition rules and neighborhood structure for the CA (e.g., Moore or von Neumann). The C++ data structure will be designed to efficiently represent terrain cells and water depth.
P2.2	GIS Data Reading Implementation	Phase 2	Develop the C++ module for correctly reading and parsing the input GIS data (Digital Elevation Model or DEM). This module must initialize the initial state of the Cellular Automata based on the topography and the flood starting point.
P2.3	Water Propagation Rules (CA)	Phase 2	Code the core of the simulator, which includes the equations and logic for water transfer between cells in each iteration. This represents the underlying flow model that governs the flood simulation.
PHASE 3: PARTIAL DELIVERABLE			
P3.1	Unit Testing of the CA Engine	Phase 3	Conduct exhaustive testing of critical code functions, ensuring that GIS data reading, terrain initialization, and, mainly, the CA water propagation rules behave as expected. Focus will be on identifying precision and performance errors.
P3.2	Integration and Data Validation	Phase 3	Run the complete simulator with a representative set of GIS data. Simulation results will be compared against known flood data or other model outputs to validate the program's behavior.
P3.3	Progress Document Generation (Part 1)	Phase 3	Compile and draft the initial chapters of the FMP document in LaTeX. This includes the introduction, literature review, and a detailed description of the development methodology and the Cellular Automata design.
PHASE 4: FINAL DELIVERABLE			

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

ID	Activity	FMP Phase	Description (2-5 Lines)
P4.1	Results Analysis and Visualization	Phase 4	Process the simulation output data (depth maps, flooding time, etc.). External tools or C++ libraries will be used to generate clear graphs and visualizations that allow the interpretation of the flood impact.
P4.2	Final Thesis Writing and Revision	Phase 4	Complete the Results and Conclusions chapters of the LaTeX document. A full review of style, grammar, and formatting will be performed to ensure compliance with academic standards.
P4.3	Oral Presentation Preparation	Phase 4	Design and create the presentation slides. The speech will be rehearsed, emphasizing the justification of the CA model, the C++ development process, and the key results obtained with the GIS data.
PHASE 5: PRESENTATION AND VIRTUAL DEFENSE			
P5.1	Final Supervisor Review	Phase 5	Final session with the FMP supervisor to address last-minute corrections to the document, code, and presentation. Minor details will be adjusted before the final submission and defense.
P5.2	Virtual Defense	Phase 5	Formal presentation of the FMP to the evaluation committee. Technical decisions, methodology, and project conclusions will be defended.

1.5 Document Structure

This thesis is divided into five parts that help the reader understand how the distributed flood simulation system was planned, designed, built and tested. Here is what you will find in the following parts:

- **Chapter 2**: State of the Art provides a comprehensive review of existing scientific literature regarding flood simulation. It focuses on the mathematical foundations of Cellular Automata (CA), current C++ flow models, and the state of distributed systems in environmental modeling.
- **Chapter 3**: Problem Description and Proposal details the technical design of the solution. This

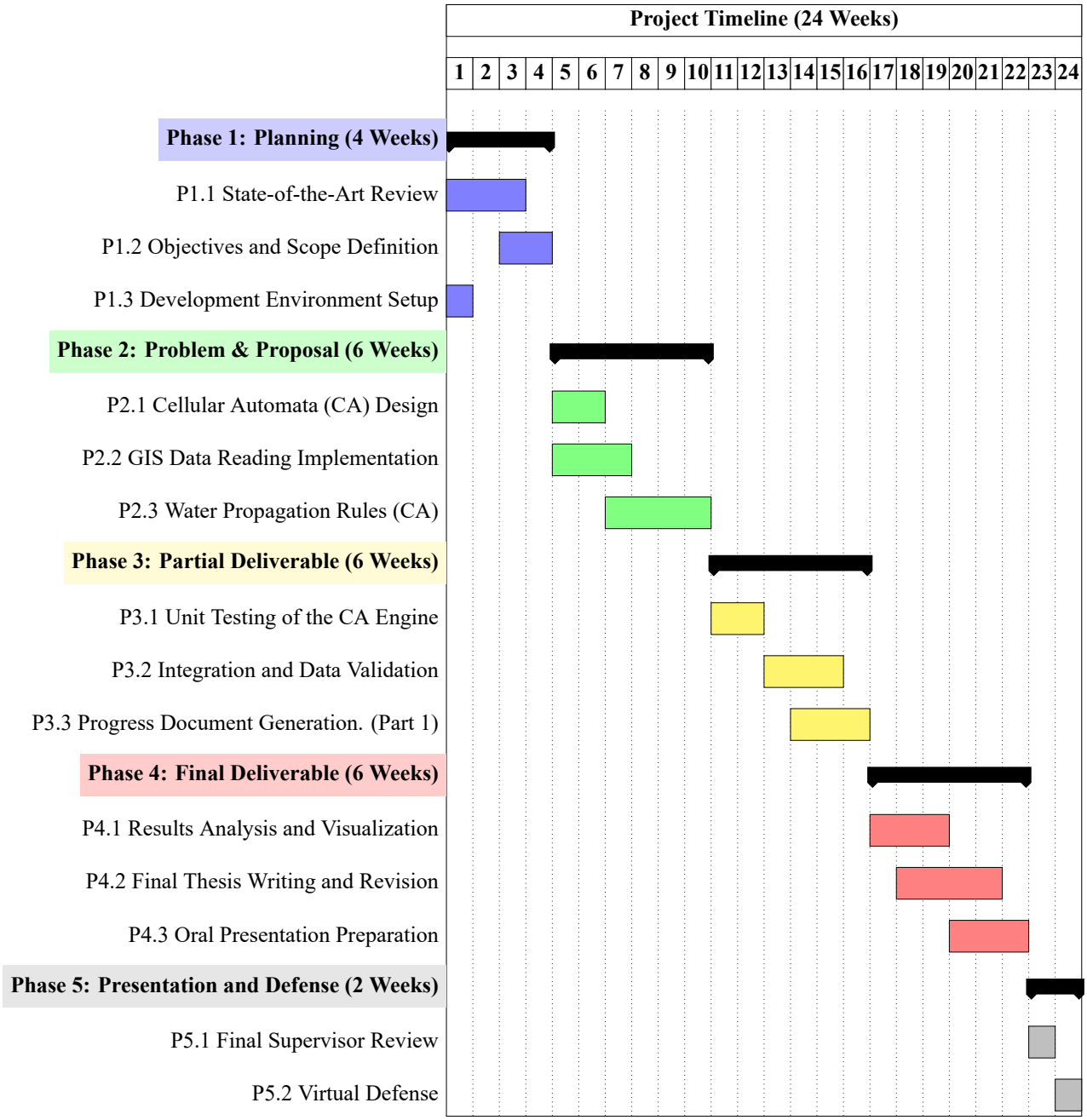

Figure 1.1: Project Timeline (24 Weeks)

includes the definition of the CA transition rules and neighborhood structures, the methodology for parsing GIS data (DEM), and the architecture of the distributed system using the MQTT protocol for real-time communication.

- **Chapter 4**: Implementation and Results describes the technical realization of the simulator in C++. It covers the development environment setup, the execution of unit and integration tests, and the final simulation of the Valencia DANA event. This chapter also includes the

visualization and analysis of the resulting flood maps and impact data.

- **Chapter 5**: Conclusions and Future Work offers a critical reflection on the project's findings. It evaluates the simulator's performance and accuracy against the established objectives and proposes future research lines, such as the full integration of the system into a real-time Digital Twin framework.

Chapter 2

State of the Art

2.1 Current Approaches in Flood Modeling

2.1.1 Overview of Flood Simulation Paradigms

Flood modeling has changed a lot in the ten years. It used to be about looking at data to make predictions. Now it is about making predictions in time. This evolution is driven by the increasing frequency of extreme meteorological events, such as the catastrophic DANA experienced in the Spanish Mediterranean in October 2024. People wrote about this in papers by Beltran et al., 2025 and Muñoz et al., 2025. There are three ways that people make flood models now. The first way is to use based hydrodynamic models. The second way is to use computational models, such, as Cellular Automata. The third way is to use something called Digital Twins, which's a new idea that is part of something bigger called Society 5.0. Digital Twins are a new way to do it.

2.1.2 Hydrodynamic Modeling and Shallow Water Equations (SWE)

The traditional gold standard in flood simulation uses the Shallow Water Equations (SWE) to model floods. These equations come from the Navier-Stokes equations. Assume that the horizontal length is much bigger than the vertical depth.

Advanced methods like Godunov-type schemes are often used to solve these equations on meshes. As noted by Aleksyuk and Belikov, 2017, these schemes are important for handling terrain with areas where water gets shallow and the bottom changes suddenly. In areas the water flow can change from slow to fast and vice versa. The models use solvers to capture shock waves, which is crucial for simulating dam breaks or violent flash floods in steep areas.

Recently these models have been used in places like the Northern Caucasus as discussed by Vasil'eva et al., 2021. They work with automated weather and water monitoring systems. These systems allow for simulations in small river areas where flash floods happen quickly. However these detailed models are very costly to run and require computers or special graphics cards to work fast

enough for large areas or in real-time. The high cost is a problem, for using them widely as noted by Aleksyuk and Belikov, 2017.

2.1.3 Discrete Models: The Rise of Cellular Automata (CA)

As an alternative to the heavy computational requirements of differential equation solvers, Cellular Automata (CA) have emerged as a robust framework for flood inundation modeling. Unlike continuous models, CA operate on a discrete grid where each cell changes state based on local transition rules.

A significant milestone in this field was the development of the multilayer cellular automata ($m : n - CA^k$) by Fonseca et al., 2011. Originally applied to slab avalanche configurations, this model proved that complex geophysical phenomena could be effectively represented by separating the geographical data into distinct layers (GIS-based) and applying formal transition rules (represented via SDL - Specification and Description Language). This ”Data-as-Interface” strategy allows for a clear decoupling between the model logic and the implementation, facilitating extensibility.

Current research has further refined CA-based solvers to include dynamic-wave approximations, achieving accuracies comparable to traditional Finite Volume methods but with execution speeds up to 300(Cellular Automata fast flood evaluation) have demonstrated the ability to predict maximum inundation depths in seconds, making them ideal for exploratory water system planning and real-time emergency response (Jamali et al., 2019).

2.1.4 Digital Twins and the Society 5.0 Framework

The most recent frontier in flood management is the integration of simulation models into Digital Twin (DT) architectures. Within the vision of Society 5.0, a Digital Twin is not merely a 3D visualization but a dynamic, real-time replica of the physical environment that ingests data from IoT sensors, satellite imagery, and meteorological radars (i Casas & i Palomes, 2026).

According to i Casas and i Palomes, 2026, Society 5.0 leverages Industry 4.0 technologies—such as Big Data, AI, and Digital Twins—not just for industrial efficiency but for human-centric social benefit, specifically in disaster resilience. In this context, a flood model becomes an ”Open Model” that supports decision-making by allowing authorities to simulate ”what-if” scenarios in real-time. For instance, during the 2024 DANA event, the lack of integrated real-time modeling was cited as a major hurdle in effective emergency management. The transition toward ”Digital Twin Smart Cities” (DTSC) aims to bridge this gap by providing a continuous synchronization between the physical river basin and its digital counterpart (Mazzetto, 2024).

2.1.4.1 Distributed Communication in Early Warning Systems (MQTT)

The transition from static simulations to reactive Digital Twins requires a communication backbone capable of handling high-frequency data exchange with minimal overhead. In the context of flood monitoring, the Message Queuing Telemetry Transport (MQTT) protocol has emerged as a standard for real-time environmental telemetry (Dionísio et al., 2025). Unlike traditional request-response models (HTTP), MQTT's publish-subscribe architecture allows for a decoupled interaction between the simulation engine and the visualization layer. This is particularly critical in flash flood scenarios where sub-second latency in data dissemination can significantly impact the efficacy of emergency response.

2.1.4.2 Software Design Patterns for Scientific Modeling (Hexagonal Architecture)

A common pitfall in scientific software development is the "Big Ball of Mud", where physical logic is tightly coupled with data I/O and specific library dependencies (e.g., GIS formats or networking protocols). To ensure the longevity and testability of the simulator, this research adopts the Hexagonal Architecture (also known as Ports and Adapters), Cockburn, 2005; Nunkesser, 2022. By isolating the Cellular Automata Core (the "Inside") from the infrastructure concerns (the "Outside"), such as HDF5 file readers or MQTT publishers, the system achieves a high degree of technological agnosticism. This architectural decoupling allows for the substitution of data sources or visualization engines without modifying the underlying hydraulic transition rules. In modern scientific computing, this approach is increasingly recognized as a best practice to manage the complexity of multi-layered models ($m : n - CA^k$) and to facilitate the integration of heterogeneous geospatial data.

2.1.5 Data-Driven and Empirical Approaches

Complementing the numerical models are empirical approaches focused on peak flow estimation and hydrological response. Muñoz et al., 2025 highlight the importance of "reconstructing" flood events using post-event field data and empirical formulas (such as the Manning equation and the rational method) to validate hydraulic models. Furthermore, the use of public datasets (AEMET, SAIH, Copernicus) has become standard for pre-processing inputs, although the "spatialization" of rainfall remains a challenge in small, ungauged catchments (Vasil'eva et al., 2021).

2.2 Analysis of Shortfalls in Existing Solutions

The big improvements in modeling and discrete simulation that we talked about earlier do not change the fact that the really bad flash floods in 2024 like the DANA in the Júcar River Basin showed us that there are big problems with the way things are done now. These problems can be put into four groups:

it takes a long time to get the results, the system is not flexible, the data is not shared well, and the flash flood systems do not have a simple way for people to make decisions. The flash flood systems have these problems because of the latency, the architectural rigidity, the data integration silos and the lack of a human-centric decision-making interface, in the flash flood systems.

2.2.1 Computational Latency vs. Flash Flood Dynamics

The primary shortfall of physically-based models (SWE solvers) remains their execution time. Achieving high-precision results on complex, unstructured meshes requires high-performance computing resources that are often unavailable or too slow to deploy during the narrow ”lead time” of a flash flood. In events where the time from peak rainfall to peak flow is measured in minutes rather than hours, the computational overhead of traditional numerical methods prevents them from being used as proactive forecasting tools. Instead, they are frequently relegated to post-event forensic analysis (Muñoz et al., 2025).

2.2.2 Monolithic Architectural Rigidity

Many flood simulation platforms are built as one system. This makes it very hard to add features or change parts of it. For example, you can't just change the way water soaks into the ground or add a weather data source without overhauling everything. This lack of flexibility makes it tough to create models like those, in the Society 5.0 plan (i Casas & i Palomes, 2026). These systems also only work with specific data formats. That makes it hard for them to work with GIS tools and real-time simulation engines.

2.2.3 Simplified Representation of Spatial Heterogeneity

While standard Cellular Automata (CA) offer the speed required for real-time applications, they often suffer from an oversimplification of the physical environment. Most CA implementations treat the terrain as a single-dimensional grid, failing to represent the complex interactions between different layers of information—such as soil saturation, subterranean drainage systems, and varying urban roughness—simultaneously. The inability to handle ”multilayered” spatial data in a unified tensor-like structure limits the model's capacity to simulate the non-linear behavior of water in urbanized basins (Fonseca et al., 2011).

2.2.4 The Socio-Technical Gap in Emergency Management

A final, critical shortfall is the disconnect between the technical output of simulation models and the operational needs of emergency responders. As highlighted by regarding the 2024 DANA crisis, the

lack of a dynamic, real-time ”Digital Twin” meant that decision-makers were operating with fragmented and delayed information. Existing models do not easily integrate with the modern IoT and radar infrastructure needed to provide a continuous, synchronized digital representation of the evolving disaster.

2.3 Problem Statement

The main problem that this thesis is trying to fix is that current flood modeling systems are not good enough. They are not fast, accurate, and flexible all at the same time. This means they are not very useful for making decisions during really bad flash floods.

There is a gap between two types of models. The first type, hydrodynamic solvers, is very precise but very slow. The second type, discrete models, is fast but not very precise. This gap is made worse because we do not have a way to handle all the complex data that we have about the earth. We need a system that can handle all this data in a way that's scalable.

The 2024 DANA event showed us that our old way of predicting floods is not good enough for what society needs today. We need a way of doing things. We need systems that can take in data in time, process it quickly ,and give us results that we can use to make decisions. These systems are called Reactive Digital Twins.

So this research is trying to solve some technical problems. These problems are:

1. **Modeling Complexity**: How to create a model that can show us how different things like rain and water flowing over the ground interact with each other. We want to use a cellular automata to do this.

2. **Architectural Decoupling**: How to design a software architecture (Hexagonal/Ports and Adapters) that allows the simulation engine to remain agnostic to specific GIS data formats and storage backends.

3. **Performance and Scalability**: How to implement this logic in a high-performance environment (C++) that leverages modern data structures (Tensors) to ensure the simulation speed remains well below the real-time threshold.

If we can solve these problems then we can create a way of managing floods. This new way will be able to take in lots of data, about the environment and turn it into decisions that can save lives. Flood management tools will be much better. They will be able to help us make decisions during really bad floods.

Chapter 3

Methodology and Design

3.1 Proposed Analytical Framework

To fill the gap between what we learn from modeling and what a Reactive Digital Twin really needs, this research suggests a new way of looking at things. The plan is not about simulating water but about creating a system that can handle a lot of changing data like what happens during a flash flood, such as the 2024 Valencia DANA.

The new approach is based on three ideas that make sure the system is strong and accurate:

1. **Mathematical Formalization of Discrete Space**: Utilizing the $m : n - CA^k$ paradigm to represent complex, non-linear hydraulic interactions through discrete transition rules. This method is faster than ways of solving Shallow Water Equations.

2. **Technological Decoupling (Hexagonal Architecture)**: Implementing a strict separation between the simulation core and external concerns (GIS data, MQTT messaging, logging). This ensures that the physical logic remains pure and adaptable to different hardware or data sources.

3. **Distributed Telemetry and Interoperability**: Designing a communication layer that transforms the simulator from a standalone tool into a distributed node. By using MQTT, the system enables real-time synchronization between the computational engine and remote visualization or decision-support interfaces.

By putting these ideas we can move away from the old way of doing simulations, batch-processing, where we do one thing at a time. Instead we can do everything at the time: get data process it and share the results all within a system that is connected and working together. That is what the Reactive Digital Twin is all about.

3.1.1 The Multilayer Conceptualization ($m : n - CA^k$)

The cornerstone of the simulation logic is the adoption of the Multilayer Cellular Automata ($m : n - CA^k$) formalism. Traditional Cellular Automata (CA) typically operate on a single state per cell,

which is insufficient for representing the heterogeneous variables of a river basin. The $m : n - CA^k$ generalization allows for a multidimensional representation where the environment is discretized into a stack of n computational layers.

Mathematical Formalization

Formally, the simulation environment is defined as a specialized cellular space where each cell c at a given coordinates (i, j) is characterized by a state vector $S_{i,j}$. The system is represented by the tuple:

$$A = \langle R, E, V, N, \Psi \rangle$$

Where:

- R is the m-dimensional cell space.
- E is the set of n layers that define the global state and the physical environment of the system.
- V represents the vicinity function.
- N represents the nucleus function.
- Ψ is the combination or evolution function that governs the fluid dynamics.

3.1.1.1 Cell Space (R) and Layer Definition (E)

The simulation space R is defined on a regular two-dimensional grid $R \subset \mathbb{Z}^2$, where each cell is identified by its spatial coordinates (i, j). The physical size of each cell is determined by the spatial resolution Δx.

The system consists of a set $E = \{E_1, E_2, E_3, E_4\}$ of $n = 4$ layers. One of the fundamental characteristics of the $m : n - CA^k$ model is that the layers may differ in their geometric and temporal dimensions. Through these tensor representations, they are defined as follows:

1. E_1: Topobathymetry ($Z_{topo-bathy}$). A static 2D layer of dimension $X \times Y$. It combines the Digital Elevation Model (DEM) with the bathymetry of the watercourses, providing the absolute elevation of the riverbed: $E_1 \subset \mathbb{R}^{X \times Y}$.
2. E_2: Land Cover (LC). A static 2D layer of size $X \times Y$. It identifies land uses, which are algebraically mapped to Manning roughness coefficients (μ_{soil}) via a state transfer function: $E_2 \subset \mathbb{N}^{X \times Y} \xrightarrow{f} \mathbb{R}^{X \times Y}$.
3. E_3: Water Depth (WD). A 2D dynamic layer of dimension $X \times Y$. It represents the main state of the cell at time t, storing the water depth in meters: $E_3 \subset \mathbb{R}^{X \times Y}$.

4. E_4: Rainfall (RF). A 3D dynamic layer of dimension $X \times Y \times T$, explicitly incorporating the time domain to provide the rainfall rate for each step Δt: $E_4 \subset \mathbb{R}^{X \times Y \times T}$.

3.1.1.2 Vicinity Function (V) and Nucleus Function (N)

The **Vicinity Function** (V) determines the subset of adjacent cells that influence the calculation of the future state of a given cell. A Moore neighborhood with radius $r = 1$ is adopted, defining a set of 8 adjacent cells connected orthogonally and diagonally.

$$V(i, j) = \{(i + u, j + v) \in R \mid \max(|u|, |v|) \leq 1\}$$

The **Nucleus Function** (N) is projected onto the central cell itself to update layer E_3, mapping the spatial modifications through local rules: $N(i, j) = \{(i, j)\}$.

3.1.1.3 Intra-layer Transitions: Horizontal Propagation Dynamics

Intra-layer transitions define the kinematic behavior of surface water flow across the discretized spatial grid. While inter-layer transitions govern vertical fluxes (such as precipitation and infiltration), this section focuses on horizontal propagation, where the state of a cell at time $t + 1$ is determined by the hydraulic interactions with its immediate neighborhood at time t.

A. Neighborhood Formalization and Hydraulic Gradients

Following the technical implementation in C++ and TensorFlow, the system employs a Moore neighborhood ($k = 8$). This choice allows for a more realistic flow propagation that is significantly less dependent on grid orientation compared to the simpler von Neumann neighborhood.

For every cell $c_{i,j}$, the simulation engine computes a vector of hydraulic gradients S toward its eight neighbors. The gradient is defined by the difference in potential energy (terrain elevation H plus water depth WD) divided by the Euclidean distance L_k:

$$S_k = \frac{(H_{i,j} + WD_{i,j}) - (H_{neighbor,k} + WD_{neighbor,k})}{L_k}$$

Where L_k corresponds to the cell size for cardinal neighbors and $\sqrt{2} \times$ cell_size for diagonal neighbors. Adhering to the physical logic of the model, only positive gradients ($S_k > 0$) are considered, ensuring that water only moves toward cells with lower hydraulic potential.

B. Velocity Estimation via Manning's Equation

Flow velocity in each of the eight directions is derived from a simplified version of Manning's Equation, adapted for Cellular Automata environments. The model integrates surface friction through a

roughness coefficient n (Manning's n), which is dynamically retrieved from a Lookup Table (LUT) based on land cover classification.

The velocity V_k in direction k is calculated as:

$$V_k = \frac{1}{n} \cdot R^{2/3} \cdot \sqrt{S_k}$$

In this context, R represents the hydraulic radius, which for shallow laminar flows—typical of the Valencia DANA event—is approximated by the water depth (WD). This relationship allows the simulator to accurately capture how urban infrastructure or dense vegetation in the Valencian basin acts as a natural brake on the flood wave.

C. Multiple Flow Direction (MFD) and Mass Balance

In contrast to traditional Single Flow Direction (D8) models, this simulator implements a Multiple Flow Direction (MFD) algorithm. The total outflow volume from a cell is distributed among all neighbors with a lower potential, weighting the amount of water sent to each neighbor according to the magnitude of their respective gradients.

The final mass balance for the surface water layer is governed by the following discrete differential equation:

$$WD_{i,j}^{t+1} = WD_{i,j}^{t} + \left(\frac{\sum \Delta V_{in} - \sum \Delta V_{out}}{Area_{cell}} \right)$$

To ensure numerical stability and prevent artificial oscillations (particularly in areas with steep slopes), the implementation includes a precision filter (ϵ) and a flow-limiting constraint: the total outflow can never exceed the volume of water physically available in the cell during the current time step.

D. Tensorial Implementation and Optimization

From a software engineering perspective, these transitions are not computed through iterative cell-by-cell loops. Instead, the model utilizes tensorial shift operations (leveraging padding and slicing). By representing the river basin as a multidimensional tensor, the simulator calculates the eight gradients for the entire Valencia grid simultaneously. This approach fully exploits CPU/GPU vectorization capabilities, ensuring that total computation time remains well below the real-time threshold required for emergency response.

3.1.1.4 Inter-layer Transitions: Vertical Propagation Dynamics

Inter-layer transitions govern the vertical interactions between the different information layers within a single cell $c_{i,j}$. Unlike horizontal propagation, these processes are local to the cell's vertical column and represent the exchange of mass and properties between the atmospheric, surface, and sub-surface layers.

A. Precipitation Injection (Atmosphere-to-Surface)

The primary driver of the simulation, particularly in the context of the Valencia DANA, is the localized and intense rainfall. In the $m : n - CA^k$ framework, precipitation is treated as a dynamic input layer P that contributes directly to the water depth layer (WD).

For each simulation step Δt, the water depth is updated to account for the incoming volume:

$$WD^*_{i,j} = WD^t_{i,j} + \left(R^t_{i,j} \cdot \Delta t\right)$$

Where $R^t_{i,j}$ represents the rainfall intensity (mm/h converted to m/s) at that specific coordinate.

B. Land Cover and Hydraulic Parameter Mapping

A critical inter-layer process in the proposed model is the transformation of discrete geospatial data into continuous physical parameters. As implemented in the simulation core, the Land Cover layer (static GIS data) acts as a descriptor for the surface roughness.

The system utilizes a Lookup Table (LUT) approach within the preprocessing stage to map land-use categories (e.g., urban fabric, orchards, industrial zones) to specific Manning's n coefficients. This mapping is formally defined as:

$$\forall c_{i,j} \in \mathcal{L}_{LandCover} \implies n_{i,j} = \text{LUT}(category_{i,j})$$

This ensures that when the intra-layer propagation logic (Manning's Equation) is executed, it correctly interprets that water moves slower through the dense urban streets of Paiporta or Sedaví compared to the smooth paved surfaces of the V-31 highway, capturing the hydraulic complexity of the Valencian metropolitan area.

C. Tensorial Data Integration

From a data engineering perspective, these inter-layer transitions are optimized using element-wise tensor operations. By aligning all layers (Precipitation, Land Cover, Water Depth) into a single multi-channel tensor, the vertical update for millions of cells is performed in a single computational pass. This "Data-as-Interface" philosophy allows the $m : n - CA^k$ engine to maintain high throughput, as

vertical interactions are computed as simple addition or multiplication operations across the tensor's depth axis.

3.1.1.5 Boundary Conditions: The Impermeable Wall Logic

The handling of the grid limits is essential to maintain the physical integrity of the simulation. In this model, boundary conditions are implemented using a Wall-Constraint strategy through specialized tensor padding.

A. Topographic Symmetric Padding

To define the spatial limits of the simulation, the Topography layer (H) utilizes Symmetric Padding. This mirrors the elevation values at the edges of the grid:

$$\text{Padding}(H, \text{mode='SYMMETRIC'})$$

By mirroring the elevation, the engine ensures that there is no artificial "cliff" at the edge of the data, maintaining a consistent gradient calculation even for cells located at the absolute border of the study area.

B. The Impermeable Boundary (Wall)

The physical behavior of the boundary is configured as a Wall (Impermeable Boundary). This is achieved by combining the symmetric topography with a Constant Zero-Padding for the water depth layer:

- **Topography**: Mirrored to prevent gradient artifacts.
- **Water Depth**: Padded with 0.0 at the exterior.

This configuration forces the transition logic to treat the exterior as a non-interactive zone. Because the engine calculates flow based on gradients, repeating the topography at the edge effectively creates a "barrier" where water cannot naturally escape unless a significant hydraulic head is reached, simulating a contained environment. This "Wall" logic is critical for modeling urban sub-basins in the Valencia DANA scenario where surrounding infrastructure (highways, railways, or embankments) acts as a physical limit to the flood's expansion.

3.2 Software Architecture: The Hexagonal Approach

One big problem in computing is that the physical models are really closely tied to the underlying infrastructure. This includes things like the formats we use to store data, the ways that computers

talk to each other and the special libraries that are made for types of hardware. In a lot of hydraulic simulators the rules for how things change are all mixed up with the code that reads specific types of files like GeoTIFF or CSV files. This makes it really hard to keep the system running test it or make changes to it.

To get around these problems, this research uses the Hexagonal Architecture or the Ports and Adapters pattern. The main goal is to keep the Simulation Core separate from other things that might affect it. By making a line between the ”domain logic”, which is just the physics of how floods work; and the ”infrastructure”, which is how we get and send data. The simulator becomes a highly flexible and technologically agnostic engine.

This way of designing the system is especially important, for a Reactive Digital Twin. It lets the system switch from reading data from HDF5 files to getting real-time data from MQTT rainfall streams without changing any of the code that controls how the hydraulic system changes. The hydraulic Simulation Core stays the same.

3.2.1 Core vs. Infrastructure

The Hexagonal design divides the system into two distinct regions: the Inside (The Core) and the Outside (The Infrastructure). The fundamental constraint governing this architecture is the Dependency Rule, which states that dependencies must always point inwards toward the Core.

A. The Core (The ”Inside”)

The Core represents the ”Hexagon” itself. It contains the pure business and scientific logic of the project. In this thesis, the Core is composed of:

- **The** $m : n-CA^k$ **Engine**: The implementation of the transition functions (Manning's equation, MFD algorithm) as defined in Section 3.1.

- **The Domain Model**: The tensorial representation of the grid, cell states, and hydraulic parameters.

- **Application Services**: Orchestrators that handle the simulation loop and time-step synchronization.

The Core has no knowledge of the outside world. It does not know if the data comes from a local disk or a cloud-based broker; it simply defines the interfaces it needs to operate.

B. The Infrastructure (The ”Outside”)

The Infrastructure consists of everything that interacts with the Core. It is responsible for the ”technical details” that the Core ignores. This includes:

- **Data Sources**: HDF5 file handlers, GIS raster readers, and SQLite databases.
- **State Computation**: High-performance execution engines for tensor operations.
- **Communication**: MQTT clients for real-time telemetry and JSON serializers for web-based visualization.
- **Persistence**: Logging systems (e.g., Loguru) and state-exporting tools.

By segregating these concerns, we ensure that technological obsolescence in the infrastructure (e.g., a protocol update or a change in GIS standards) does not jeopardize the scientific validity of the simulation engine.

3.2.2 Port and Adapter Specification

To bridge the gap between the Core and the Infrastructure, the architecture utilizes Ports (interfaces) and Adapters (implementations).

A. Driver Ports (Input)

Driver Ports define how the external world ”triggers” or ”feeds” the simulation. They represent the entry points into the Hexagon.

B. Driven Ports (Output/Supporting)

Driven ports are interfaces used by the Core to delegate tasks to external tools or specialized hardware.

3.2.3 Technical Advantage: Testability and Mocking

A significant advantage of this specification is the ability to perform Unit Testing on the hydraulic logic. Because the Core is decoupled from the hardware and files, we can create ”Mock Adapters” that feed the Core with pre-defined, small-scale tensors to verify the mathematical correctness of the $m : n - CA^k$ transitions without needing a full GIS environment or an active MQTT broker.

3.3 Distributed Communication Model (MQTT)

The effectiveness of a flood emergency system depends not only on the precision of the calculation but also on the speed and reliability of data delivery. The $m : n - CA^k$ engine utilizes the MQTT (Message Queuing Telemetry Transport) protocol, a lightweight M2M (machine-to-machine) communication standard, to bridge the gap between the high-performance C++ core and remote visualization or decision-support clients.

3.3.1 Decoupling Strategy: Publisher-Subscriber Paradigm

To maintain the rigorous performance requirements of the simulation core, the communication layer is built upon the Publisher-Subscriber pattern. This strategy ensures a total decoupling between the ”Model” (the simulator) and the ”View” (the dashboard).

- **Anonymity and Scalability**: The simulator (Publisher) remains unaware of the number, location, or nature of the subscribers. This allows multiple emergency units to monitor the Valencia DANA simulation simultaneously without adding computational overhead to the C++ core.

- **Space and Time Decoupling**: The simulation can continue its calculation cycles even if the network is temporarily unstable or if the visualization clients are busy, as the MQTT broker acts as an intermediate buffer.

3.3.2 Topic Hierarchy and Payload Design

The system organizes information through a structured Topic Hierarchy, which serves as a logical routing map for telemetry. Following the implementation in the MqttOutput adapter, the hierarchy is rooted in the scenario name to support multi-event management:

- FloodSim/scenarioName/system/handshake: Used for the initial handshake, broadcasting grid dimensions ($size\_x, size\_y$), cell resolution, and simulation start times.

- FloodSim/scenarioName/events: The primary high-traffic channel used for broadcasting dynamic updates of water depth and flood risk levels.

- FloodSim/scenarioName/control/events: A bi-directional channel for frame acknowledgments (Acks).

Lightweight JSON Payload Design

Data efficiency is achieved through a specialized JSON Serialization strategy. Rather than sending the entire $m : n - CA^k$ state in every message—which would saturate the bandwidth—the system uses a "Differential Update" philosophy. The JsonSerializer generates discrete process-oriented messages:

- **Lifecycle Messages**: System_Ping, InitMap_Config, and SimEnd.
- **State Messages**: EYE_SetState_Layer containing only the modified cells, indexed to the global grid.

3.3.3 Synchronicity and Latency Management

Broadcasting high-resolution grids (e.g., the Valencia metropolitan area) requires a sophisticated strategy to prevent network congestion. The implementation handles this through three mechanisms:

A. Data Chunking and Batching

To avoid exceeding the Maximum Transmission Unit (MTU) of the network or the broker's buffer limits, the system implements Grid Chunking. A single simulation frame is divided into manageable chunks (e.g., $40,000$ cells per message). These chunks are sent in controlled batches (e.g., 10 chunks per batch), allowing the network to breathe between bursts of data.

B. The Frame Orchestration Protocol

To ensure the subscriber assembles a consistent view of the flood, a strict orchestration sequence is followed for every simulation step t:

1. **FrameStart**: Signals the beginning of a new time step and the number of expected chunks.
2. **EYE_SetState_Layer**: Multiple messages containing the actual hydraulic data.
3. **FrameEnd**: Notifies the client that the data for time t is complete.
4. **EYE_Frame_Sync**: A synchronization heartbeat that aligns the simulation's internal clock with the visualizer's display.

C. Quality of Service (QoS) and Congestion Control

The system utilizes QoS Level 1 (At least once delivery) for critical control and synchronization messages. This guarantees that the visualizer never misses a "Frame Sync" or "Config" message. For the high-frequency hydraulic data, the implementation utilizes an Asynchronous Ack-back mechanism;

the simulator waits for a ”Chunk Ack” from the broker/client before proceeding with the next batch, ensuring that the simulation speed does not overwhelm the subscriber's processing capacity.

3.4 Data Engineering and Tensor Representation

The efficiency of a Cellular Automata simulation is fundamentally constrained by its data access patterns. While traditional GIS software operates on various file formats and coordinate systems, a high-performance simulator requires a unified, contiguous, and multidimensional memory layout. This section details the ”Data-as-Interface” philosophy, where geospatial information is abstracted into tensors to enable vectorized computation.

3.4.1 The Hierarchical IDRISI Format (HIF)

To manage spatiotemporal data (e.g., hourly rainfall or dynamic water levels) without relying on complex binary containers like HDF5 or NetCDF, this project introduces the Hierarchical IDRISI Format (HIF). The HIF is a structural specification designed to store 3D data cubes using the native 2D IDRISI raster standard as a building block.

3.4.1.1 Conceptual Structure

The HIF treats time-series data as a discrete sequence of temporal frames. Each frame is stored as an independent IDRISI raster pair (.doc for metadata and .img for the data matrix), organized within a hierarchical directory tree. This allows for ”lazy loading” of simulation steps, where the engine only reads the specific frame required for the current time-step t.

3.4.1.2 Components of an HIF Dataset

An HIF dataset consists of three primary elements:

1. **Master Metadata (.hif)**: A global descriptor file that defines the spatial context (extent, resolution, and CRS) shared by all frames in the sequence. It serves as the entry point for the simulator.

2. **Temporal Attribute Manifest (attrs.json)**: A specialized metadata file that maps simulation time-steps to specific file paths. It contains the downgrade factors, ISO-8601 timestamps, and the ordered list of frame filenames.

3. **Frame Repository**: A flat collection of .img files, each representing a single point in time, without accompanying .doc files.

By adopting this format, the system achieves a balance between the simplicity of ASCII-based/flat-binary rasters and the requirement for organized, high-dimensional temporal data.

3.4.1.3 Physical Storage and Directory Topology

The physical realization of an HIF dataset is organized as a self-contained directory that encapsulates both the global spatiotemporal metadata and the individual raster frames. This structure is designed to facilitate modular access: the simulation engine can parse the top-level manifest to understand the temporal extent and subsequently perform random access to specific time-steps without the need to scan the entire dataset.

This tree topology illustrates the resulting hierarchy after the Data Pipeline exports a dynamic layer (e.g., rainfall).

```
output/rainfall/
├── rainfall.hif
├── attrs.json
├── 000000.img
├── 000001.img
└── ...
```

Figure 3.1: Hierarchical IDRISI Format (HIF) internal directory structure visualized as a tree topology.

3.4.2 The GIS-to-Tensor Pipeline: From Rasters to Multi-Channel Cubes

The transition from environmental raw data to a simulation-ready state follows a specialized ETL (Extract, Transform, Load) pipeline. This process ensures that diverse layers (topography, land cover, and rainfall) are spatially and temporally aligned before the first simulation step occurs.

A. Extraction and Spatial Alignment

Raw data for the Valencia region—including the Digital Elevation Model (DEM) and Land Use maps—often arrives in heterogeneous formats (e.g., HIF, IDRISI) and different spatial resolutions. The pipeline utilizes a Geospatial Abstraction Layer to perform:

- **Coordinate Reference System (CRS) Standardization**: Projecting all layers to a common metric system (UTM 30N) to ensure Euclidean distance calculations in the Moore neighborhood are valid.

- **Resampling and Resizing**: Aligning all rasters to a common grid extent ($H \times W$) using bilinear interpolation for continuous data (elevation) and nearest-neighbor for discrete data (land cover).

B. Tensorization: The Multi-Channel Approach

Once aligned, the data is "tensorized." In this research, the river basin is represented as a 3D Tensor of shape (C, H, W), where:

- C **(Channels)**: Represents the n information layers of the $m : n - CA^k$ model (e.g., Channel 0: Water Depth, Channel 1: Manning's n, Channel 2: Topography).

- H, W **(Height, Width)**: Represents the discretized spatial grid.

By packing the data into this structure, the simulation engine can treat the entire basin as a single mathematical object. This allows the transition rules to be implemented as element-wise operations and matrix shifts, bypassing the high overhead of traditional cell-by-cell loops and enabling the use of SIMD (Single Instruction, Multiple Data) optimizations.

3.4.3 Data Pipeline Design (Pre-processing)

The core of any flood simulation system lies in its ability to process vast amounts of geospatial data accurately and efficiently. The Data Pipeline is designed as a centralized orchestrator responsible for the ingestion, transformation, and provision of the physical parameters required by the hydraulic model.

The primary objective is to decouple data acquisition from the simulation logic. By doing so, the system ensures that changes in data formats or sources do not affect the stability of the simulator's core. This framework is built upon three fundamental pillars:

1. **Integrity**: Ensuring that spatial coordinates and resolutions remain consistent across all data layers.

2. **Scalability**: Allowing the inclusion of new physical variables (e.g., soil moisture or infiltration) without redesigning the existing architecture.

3. **Automation**: Minimizing manual intervention during the preprocessing of raster datasets.

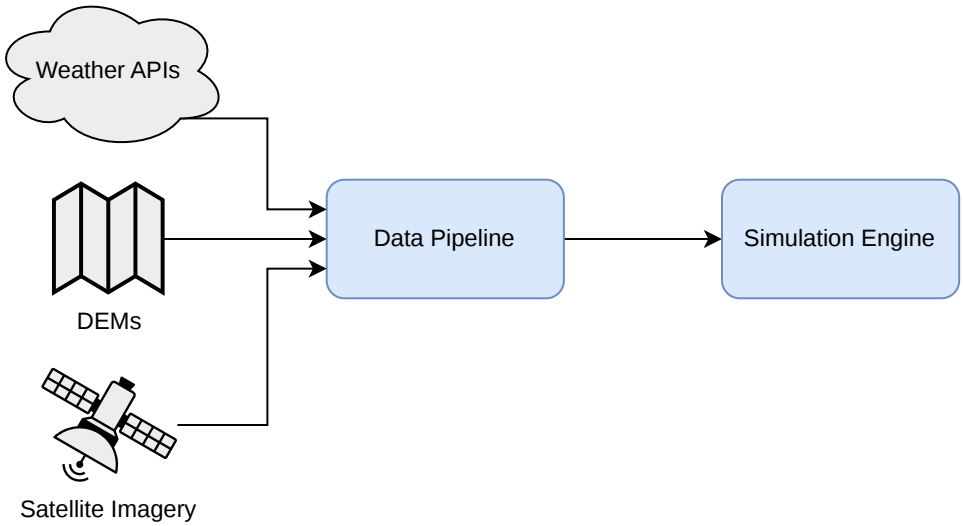

Figure 3.2: High-Level System Context

3.4.3.1 Data Categorization: Static vs Dynamic Layers

To optimize processing and memory management, the methodology categorizes geospatial information into two distinct types based on their temporal behavior during a simulation:

- **Static Layers**: These represent physical characteristics that remain constant throughout the simulation event. Examples include the Digital Elevation Model (DEM) and Land Use (Roughness). These layers are processed once and serve as the immutable ”canvas” of the terrain.

- **Dynamic Layers**: Input data that informs the transition function.These involve variables that change over time, requiring a temporal dimension in their representation. Examples include Rainfall Intensity. The design must account for the sequential reading or generation of these files, ensuring that the simulator receives the correct ”snapshot” for each time step.

This conceptual separation allows the pipeline to apply different I/O strategies for each type, significantly improving the efficiency of the data flow.

3.4.3.2 Architectural Design Patterns

To achieve a balance between flexibility and rigorous data handling, the system architecture is grounded in three fundamental software engineering patterns: Abstraction, Inheritance, and the Registry Pattern.

Abstraction and Encapsulation

The methodology treats every geospatial data source as a specialized object rather than a simple file path or a raw array. By abstracting the complexities of coordinate systems, pixel resolutions, and file formats into dedicated classes, the pipeline creates a unified interface. This encapsulation ensures that the rest of the simulator does not need to understand the internal structure of the data; it only interacts with high-level methods to retrieve physical values.

Hierarchical Inheritance (Base Classes)

The design utilizes a hierarchical structure to manage the commonalities and differences between data types.

- **Base Components**: A set of abstract base classes defines the ”contract” that all specific layers and Input/Output (I/O) handlers must fulfill.
- **Specialization**: This allows the system to distinguish between the static nature of a Digital Elevation Model (DEM) and the time-varying nature of rainfall intensity, while still sharing the same underlying logic for spatial referencing and memory management.

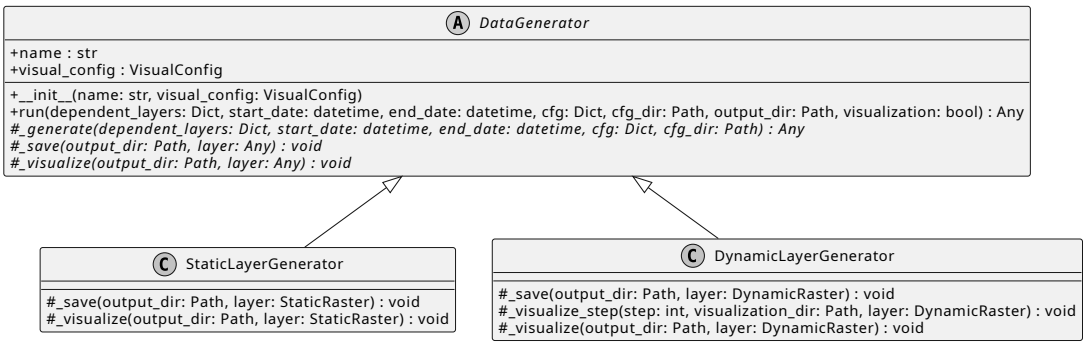

Figure 3.3: Data Generator Hierarchy

The Registry Pattern for Extensibility

One of the most critical requirements for the TFM is the ability to integrate new physical variables (layers) without modifying the core orchestrator. To solve this, the methodology adopts a **Registry Pattern**.

In this approach, the Data Pipeline acts as a central hub that does not ”know” about specific layers (like Rainfall or Roughness) at compile-time. Instead, each new layer ”registers” itself into a dictionary-like structure. When the simulation starts, the pipeline simply iterates through this registry

to initialize the required components. This design achieves Low Coupling, as adding a fifth or sixth layer only requires creating a new class that follows the established base contract, leaving the main execution logic untouched.

Decoupling I/O from Logic

Finally, the architecture enforces a strict separation between Data Access (reading/writing files) and Data Logic (processing the raster values). By isolating I/O operations into dedicated I/O Handlers, the system becomes agnostic to the storage medium. If the project were to move from local IDRISI files to a cloud-based database or a different file format (like NetCDF), only the I/O handler would need to be updated, while the layer generation logic remains intact.

3.4.3.3 Data Flow Logic

The movement of data through the system is governed by a deterministic lifecycle, ensuring that the simulation engine receives a spatially and temporally synchronized ”snapshot” of the environment. This flow is divided into four primary stages:

Configuration-Driven Discovery

The process begins with the ingestion of a centralized configuration manifest (typically in YAML format), which serves as the ”Control Plane” for the entire data pipeline. Unlike a discovery-based system, the pipeline maintains a pre-defined registry of core physical layers (e.g., Elevation, Rainfall, Roughness) essential for the simulation. The configuration file is utilized to parameterize these layers and define the global execution context.

This stage manages three critical data dimensions:

1. **Global Simulation Metadata**: It establishes the temporal boundaries of the scenario (start and end timestamps) and the workspace topology (output directory and naming conventions). This ensures that all generated layers are synchronized within the same spatiotemporal window.

2. **Layer Parameterization**: For each architectural layer, the configuration provides specific execution parameters. This includes spatial resolutions, interpolation models (e.g., Variogram types for rainfall), and geographical constraints (bounding boxes). This allows the same pipeline logic to be reused across different study areas by simply adjusting the numerical constants in the config file.

3. **Visualization Manifest**: The pipeline identifies which layers require graphical rendering for research purposes. By defining a ”visualize” list in the configuration, the system selectively

triggers the visualization engine post-generation, optimizing execution time by avoiding unnecessary plots for background layers.

By separating the layer definitions (hard-coded in the pipeline's logic for structural integrity) from their operational parameters (externalized in the configuration), the system achieves a high degree of reproducibility and scientific flexibility.

Spatial Synchronization and Metadata Validation

Before any numerical processing occurs, the pipeline performs a Spatial Integrity Check. Flood simulations are highly sensitive to spatial misalignment. The flow logic enforces a "Master Grid" approach, where secondary layers (such as rainfall or roughness) must be validated against the primary Elevation Layer (DEM).

- **Coordinate Reference System (CRS) Alignment**: Ensuring all data layers use the same projection.
- **Resolution Matching**: Verifying that pixel sizes are consistent across all rasters to avoid interpolation errors during the simulation.
- **Extent Clipping**: Ensuring that all dynamic inputs cover the exact bounding box of the static terrain.

Sequential Execution: The Static-Dynamic Split

The data flow logic branches based on the temporal nature of the data:

1. **The Static Phase (Setup)**: The pipeline triggers the generation and storage of immutable layers (Elevation and Roughness). These are processed once and held in a "Ready-to-Simulate" state.
2. **The Dynamic Loop (Runtime)**: For variables like Rainfall, the pipeline enters a cyclic flow. For every simulation time step, the logic dictates a "Fetch-Transform-Provision" cycle, ensuring that the simulator always has access to the most recent temporal state without overloading the system's memory.

3.4.3.4 Validation and Logging Strategy

The robustness of the data pipeline is sustained by a dual-layer strategy: rigorous pre-execution validation to ensure structural and spatial coherence, and comprehensive logging to provide full transparency during the transformation processes.

Structural and Dependency Validation

Before processing any raster data, the system must validate the logical structure of the simulation request. This is approached using Graph Theory principles.

- **Directed Acyclic Graph (DAG)**: The dependencies between layers (e.g., Rainfall requiring an Elevation model for clipping) are treated as nodes and edges in a graph. The methodology requires a topological sort to ensure that no circular dependencies exist and that every component is initialized only when its requirements are met.
- **Schema Enforcement**: The configuration files are validated against a predefined schema to ensure that essential parameters, such as coordinate reference systems or simulation timeframes, are present and logically consistent (e.g., start_date must precede end_date).

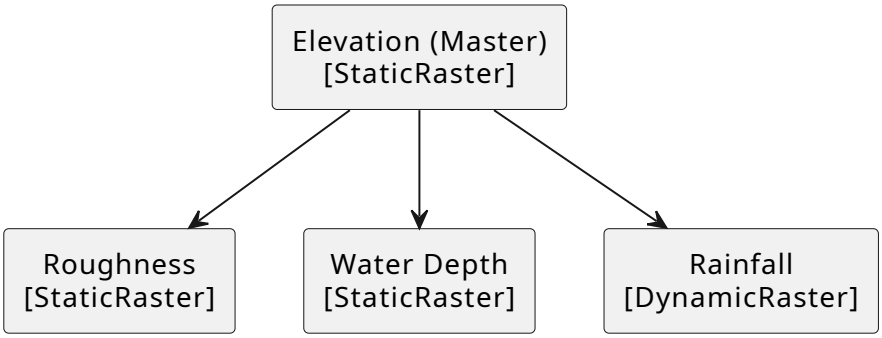

Figure 3.4: Layer Dependencies

Spatial Integrity and Physical Constraints

The core of the validation strategy lies in ensuring ”Spatial Coherence”. Since the hydraulic model operates on a numerical grid, the pipeline enforces strict spatial constraints:

- **Domain Consistency**: Validating that all input datasets overlap with the requested study area.
- **Resolution Uniformity**: Detecting and flagging discrepancies in pixel size that could lead to significant calculation errors in the fluid dynamics solver.
- **No-Data Handling**: A theoretical framework for ”Nodata” values is established, ensuring that missing geographic information does not propagate as zero-values, which would lead to physical inaccuracies in the flood extent.

Observability through Multi-Sink Logging

In long-running environmental simulations, observability is critical for both real-time monitoring and post-mortem analysis. The methodology adopts a Multi-Sink Logging Pattern:

1. **Transient Output (Standard Stream)**: Focused on high-level operational status (Success, Info, Error). It provides the user with immediate feedback on the pipeline's progress without cluttering the interface with technical debt.

2. **Persistent Output (File-based Log)**: A detailed, low-level record of every transformation, I/O operation, and minor warning. This serves as a "black box" for researchers to audit the data generation process.

3. **Severity Layering**: Messages are categorized by intensity (Trace, Debug, Info, Success, Warning, Error). This allows for granular filtering: while a researcher might only want to see successful milestones, a developer can access the "Trace" level to inspect the exact metadata of a IDRISI being processed.

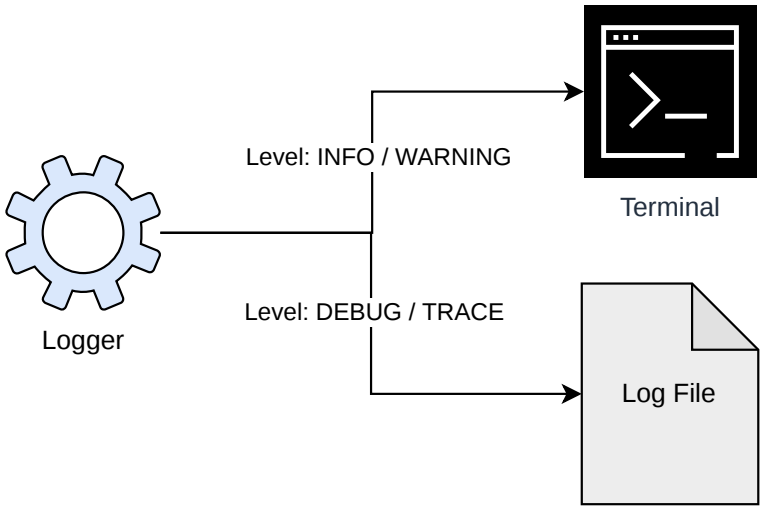

Figure 3.5: Logging Architecture

3.4.4 Temporal Layer Management: Dynamic Data Synchronization

A flash flood simulation is not a static process; it is driven by time-varying inputs, primarily precipitation. Managing these dynamic layers presents a significant challenge: loading an entire year of high-resolution rainfall data into RAM is computationally unfeasible.

A. The Temporal Update Logic

The system implements a Temporal Layer Manager within the Core to handle the lifecycle of dynamic layers. This component acts as a ”window” over the temporal axis of the data.

- **Update Frequencies**: Not all layers update at the same rate. While the topography is static, the rainfall layer might update every 10 minutes ($600s$). The manager tracks the simulation time t and triggers an update only when a specific layer's frequency threshold is crossed.

- **Hyperslab Selection**: To optimize I/O, the manager uses HDF5 Hyperslab Selection. Instead of reloading a file, it performs a direct memory-mapped read of a specific 2D slice from a 3D time-series cube $(Time, H, W)$.

B. State-Consistency and Buffer Swapping

To ensure the mathematical validity of the simulation, the Temporal Layer Manager employs a double-buffering strategy. When a new rainfall intensity matrix is loaded, it is staged in a transition buffer before being injected into the active $m : n - CA^k$ grid. This prevents ”partial updates” where one half of the grid might be using rainfall from $t = 10$ and the other from $t = 20$, maintaining strict temporal causality across the entire Valencian basin.

C. Memory Efficiency

By decoupling the ”Loading Logic” from the ”Simulation Logic,” the system maintains a constant memory footprint regardless of the duration of the simulation. This scalability is critical for the Reactive Digital Twin model, as it allows the simulator to run indefinitely, ingesting real-time MQTT data or historical HDF5 archives without degrading performance over time.

3.5 Case Study Setup: DANA 2024

The catastrophic ”Depresión Aislada en Niveles Altos” (DANA) that struck the Mediterranean coast of Spain on October 29, 2024, serves as the primary scenario for this research. This event is characterized by its extreme rainfall—exceeding 400 mm in less than 8 hours in certain areas—and the rapid, violent overflow of the Barranco del Poyo and the Turia River basin.

The complexity of the Valencian metropolitan area, with its dense urban fabric, high-speed rail embankments, and industrial zones, provides the ideal environment to validate the Hexagonal Architecture and the Tensor-based transition rules developed in this thesis.

3.5.1 Domain Discretization

The simulation domain is discretized into a high-resolution grid to capture the critical urban features (streets, underpasses, and barriers) that dictate flood propagation.

- **Spatial Extent**: The study area covers approximately $1300\ km^2$, encompassing the upper catchments of Chiva and Cheste, descending through the "ground zero" municipalities (Paiporta, Sedaví, Alfafar), and terminating at the Mediterranean coast and the Albufera lagoon.
- **Cell Resolution**: A resolution of $5m \times 5m$ per cell is adopted. This choice provides a balance between hydraulic precision and the computational efficiency required for a Real-Time Digital Twin.
- **Tensor Dimensions**: The resulting grid translates into a multi-channel tensor of approximately $5,500 \times 9,500$ cells per layer.
- **Temporal Step (Δt)**: To satisfy the Courant-Friedrichs-Lewy (CFL) stability condition in a high-velocity environment, the engine utilizes a dynamic time step, typically ranging between $1s$ and $5s$.

3.5.2 Validation Metrics

To evaluate the predictive accuracy and computational performance of the $m : n - CA^k$ engine, a dual-metric framework is established. We compare the simulated results against satellite imagery (Copernicus EMS), post-event flood marks, and historical hydrographs.

A. Spatial Accuracy Metrics (Binary Classification)

To determine how well the model identifies "flooded" vs. "non-flooded" areas, we use a contingency table approach (Hit, Miss, False Alarm).

- **Critical Success Index (CSI) / Threat Score**: Measures the spatial fit between the simulated and observed flooded areas, ignoring the "Correct Negatives" (dry areas) which can skew results in large domains.

$$CSI = \frac{Area_{hit}}{Area_{hit} + Area_{miss} + Area_{false\_alarm}}$$

- F_{score} **(F1-Measure)**: The harmonic mean of precision and recall, providing a single metric to evaluate if the model is over-predicting or under-predicting the flood extent.

B. Vertical Accuracy Metrics (Hydraulic Head)

To assess the precision of the water depth (WD) calculations, we utilize the Root Mean Square Error (RMSE). This is calculated at specific "ground-truth" points where high-water marks were recorded by the Spanish Geographic Institute (IGN) or local emergency services.

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(WD_{obs,i} - WD_{sim,i})^2}$$

C. Computational Performance (Efficiency)

Given the Reactive Digital Twin focus, a key metric is the Simulation Speed Ratio (SSR):

$$SSR = \frac{Duration_{physical\_event}}{Duration_{computational\_inference}}$$

An $SSR > 10$ is targeted, meaning the system must simulate 1 hour of the DANA flood in less than 6 minutes of real time.

Chapter 4

Implementation and Results

4.1 Software Environment and Technical Stack

4.1.1 Data Engineering Layer: Python Ecosystem

4.1.2 High-Performance Simulation Layer: C++ Environment

4.1.3 Geospatial Abstraction Libraries (Rasterio)

4.1.4 Spatio-temporal Data Management using HIF

4.1.5 Diagnostic and Observability Infrastructure

4.2 Implementation of the Data Pipeline

4.2.1 Core Architecture Implementation

4.2.1.1 The DataPipeline Orchestrator

The DataPipeline class (implemented in pipeline.py) serves as the central engine of the system. Its implementation focuses on two main tasks: dependency resolution and execution management.

As seen in the source code, the pipeline initializes by loading a YAML configuration and registering available generators (Elevation, Rainfall, etc.) into a internal registry. The execution logic follows a rigorous flow:

1. **Dependency Discovery**: It builds an adjacency list where each layer identifies its prerequisites.

2. **Topological Sort**: Using graphlib.TopologicalSorter, the pipeline converts this list into a linear execution sequence. This ensures that the Elevation layer is always processed before any layer that requires spatial clipping.

3. **Context Injection**: During execution, the pipeline passes the output of previous layers (the dependent_layers dictionary) into the next one, allowing for seamless data transfer between components without global state variables.

4.2.1.2 Base Classes and Structural Inheritance

To enforce the architectural patterns defined in Chapter 3, the implementation relies on a strict inheritance tree (found in base.py, static_layer.py, and dynamic_layer.py):

- **DataGenerator (Abstract Base Class)**: Standardizes the lifecycle of a layer through the run() method. This method acts as a template, orchestrating the generate, save, and visualize steps. This ensures that every layer, regardless of its content, follows the same operational protocol.
- **StaticLayerGenerator**: Specializes the base class for 2D data. It implements a standardized visualization routine using Matplotlib, including automated metadata boxes that display the Coordinate Reference System (CRS) and spatial resolution.
- **DynamicLayerGenerator**: Handles the added complexity of the temporal dimension. Its implementation includes logic for frame-by-frame visualization, effectively generating a temporal sequence of maps that can later be compiled into a video of the flood progression.

4.2.1.3 Data Structures and Type Safety

The implementation uses Python dataclasses (in types.py) to define the internal data representation. By using SpatialContext, StaticRaster, and DynamicRaster, the system moves away from ”primitive obsession” (using simple arrays). This ensures that every tensor is always coupled with its spatial metadata (affine transform, CRS, and dimensions), preventing the spatial misalignment errors discussed in the methodology.

4.2.2 Input/Output Management

The Input/Output (I/O) subsystem is implemented through specialized handler classes that encapsulate the complexities of geographic file formats and directory structures. By isolating these operations in IdrisiIO and HierarchicalIdrisiIO, the pipeline ensures that the core generators remain agnostic to the underlying storage drivers.

4.2.2.1 Static Raster Handling (IDRISI)

For immutable 2D layers such as elevation and roughness, the implementation relies on the IdrisiIO class. This handler manages the standard IDRISI dual-file specification:

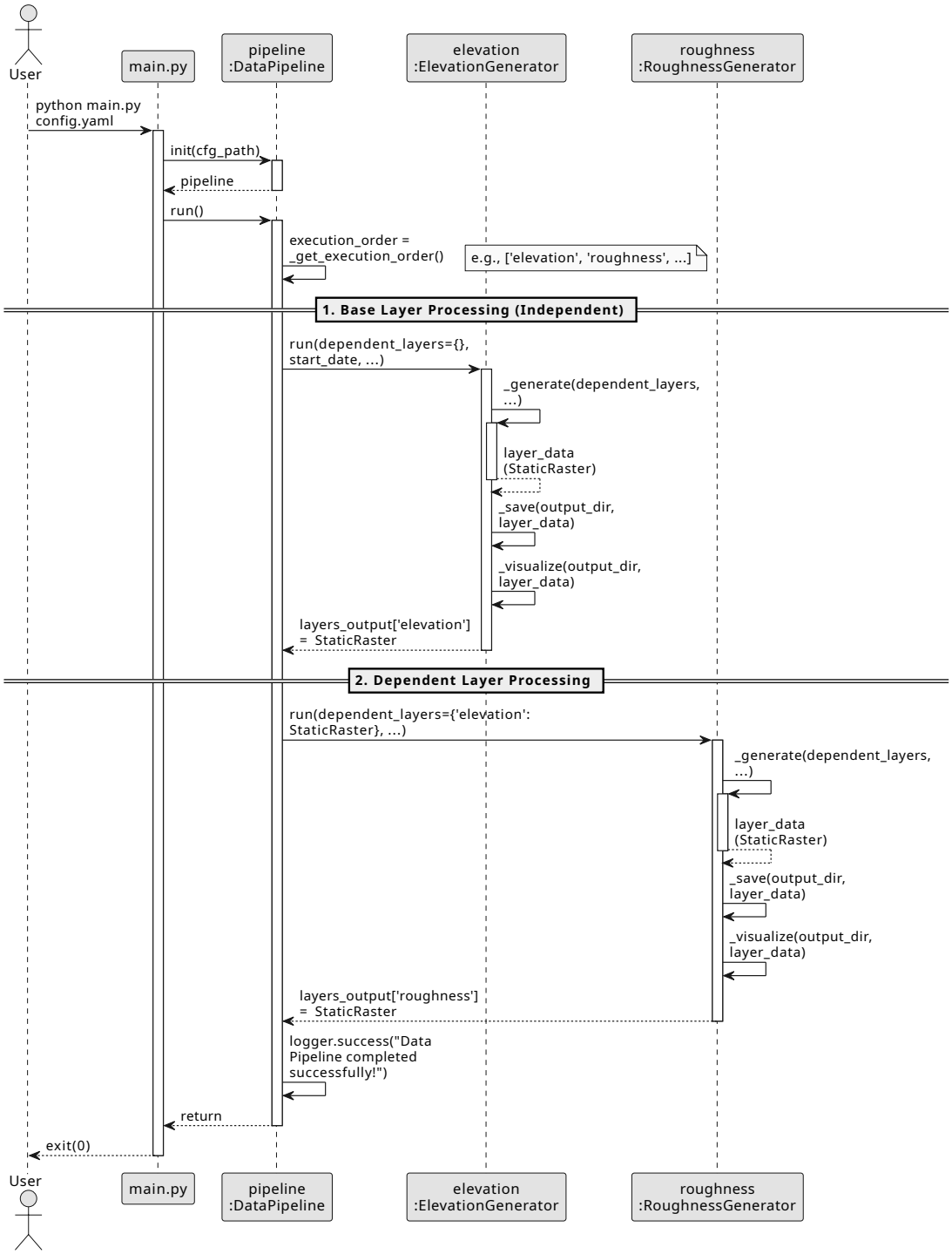

Figure 4.1: Sequence diagram of the *DataPipeline* execution flow. For visual simplicity, the diagram illustrates the iteration pattern over the first two layers of the simulator: an independent base layer (*Elevation*) and a dependent layer (*Roughness*), demonstrating the dependency injection mechanism at runtime. The rest of the layers inherit this same functional dynamic.

- **Metadata Serialization (.doc)**: The _write_doc() method programmatically generates a structured ASCII descriptor. It maps the internal SpatialContext (bounding box, CRS, cell size) to the specific fields required by the IDRISI standard, such as min. X, max. X, and posn. units.

- **Data Matrix Export (.img)**: The save() method handles the conversion of NumPy arrays into ASCII-encoded raster files. It employs a dynamic formatting strategy: integer-based layers (like land-use masks) are saved with zero decimal precision (using high-precision scientific notation to prevent data loss during the I/O process.

- **Directory Integrity**: The class automatically ensures the existence of target directories using pathlib, preventing runtime exceptions during the export phase of the pipeline.

4.2.2.2 Dynamic Raster Handling (Hierarchical IDRISI Format - HIF)

To manage spatiotemporal data (e.g., rainfall time-series) without the overhead of heavy binary containers like HDF5, the system implements the Hierarchical IDRISI Format (HIF) through the HierarchicalIdrisiIO class. This format decomposes 3D data cubes ($Time \times Y \times X$) into a structured repository of standard IDRISI rasters.

- **Master Metadata (.hif)**: The implementation generates a global descriptor file with the .hif extension. This file mimics the standard IDRISI metadata but represents the spatial context of the entire temporal sequence, serving as the primary entry point for the simulation engine.

- **Temporal Manifest (attrs.json)**: A critical component of the HIF implementation is the generation of a JSON-encoded attribute file. This manifest maps each temporal step to its corresponding ISO-8601 timestamp and physical file path. This allows the simulator to perform "Temporal Seeking"—reading only the specific frames required for a given time-step without loading the entire dataset into memory.

- **Atomic Frame Persistence**: The save() method orchestrates a loop over the temporal dimension of the data cube. For each time-step t, it delegates the serialization of the 2D slice to the IdrisiIO handler. This ensures that every individual frame is a valid, independent IDRISI dataset.

- **Format Uniformity**: By leveraging the same underlying logic for both static and dynamic layers, the I/O subsystem minimizes the dependency on third-party libraries (like h5py or NetCDF4), simplifying the deployment of the C++ simulation core.

4.2.2.3 Standardization of Spatial Metadata

Both I/O handlers enforce a unified spatial standard. During the save cycles, the pipeline ensures that all SpatialContext objects use the same coordinate units and orientation. This standardization is critical for the ”Spatial Synchronization” described in the methodology, as it prevents the simulation from running on misaligned grids.

4.2.3 Layer Generation Engines

The generation engines represent the ”concrete” implementation of the research methodology. Each engine is a Python class that inherits from either StaticLayerGenerator or DynamicLayerGenerator, ensuring that all physical layers provide a consistent set of outputs: a standardized numerical array, spatial metadata, and an automated visualization frame.

4.2.3.1 Digital Elevation Model (DEM) Engine

The Digital Elevation Model (DEM) Engine is the foundational component of the data pipeline, as it establishes the primary topographic surface upon which the hydraulic simulation and other physical layers (such as roughness or water depth) are projected. This engine is implemented in the Elevation-Generator class, which automates the acquisition and standardization of terrain data.

1. Data Sourcing and WCS Protocol

The engine utilizes the Web Coverage Service (WCS) provided by the Instituto Geográfico Nacional (IGN). Unlike standard tiled map services (WMS/WMTS) which provide pre-rendered images, the WCS interface allows the pipeline to request raw numerical elevation values (MDT) for a specific geographic extent. The implementation targets the GetCoverage operation, specifying the output format as image/tiff to ensure high-bit-depth precision of the vertical data.

2. Coordinate Transformation and Projection Logic

A critical step in the implementation is the alignment between geographic input and the simulation’s projected grid. While the user provides coordinates in decimal degrees (WGS84 / EPSG:4326), the simulator requires a planar, metric system to perform accurate physical calculations.

The engine performs a transformation to the ETRS89 / UTM Zone 30N (EPSG:25830) coordinate system using the pyproj library. This process involves:

- Projecting the South-West and North-East corners of the requested bounding box.
- Calculating the exact dimensions of the study area in meters.

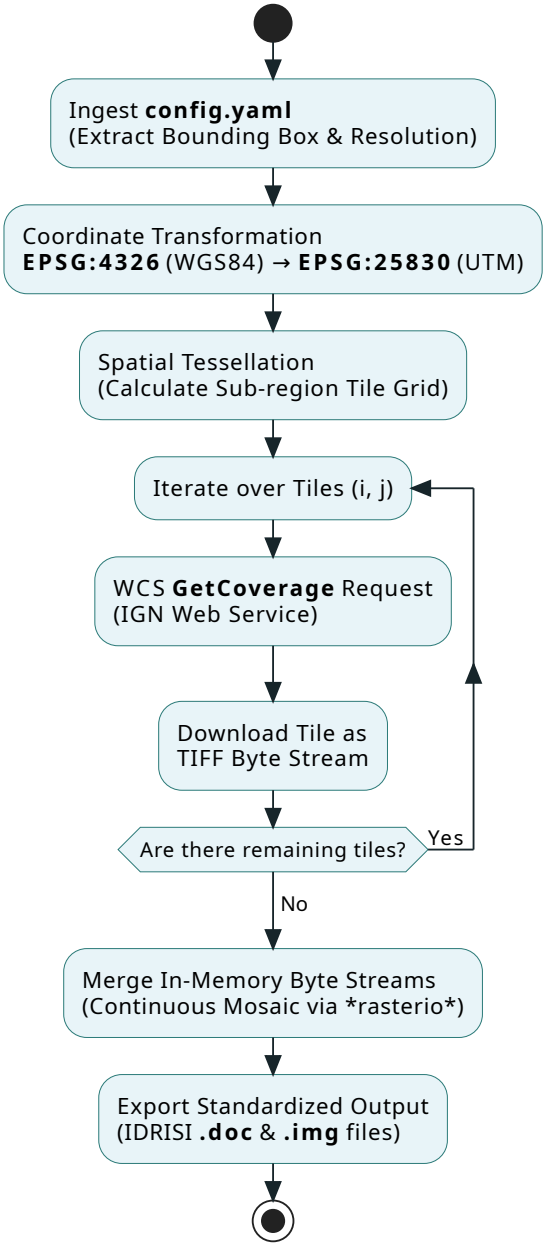

Figure 4.2: Flowchart illustrating the Digital Elevation Model (DEM) generation engine. The sequence details the pipeline execution from configuration ingestion and geographic projection mapping, through the WCS spatial tessellation strategy, to the final persistence in the IDRISI raster format.

- Computing the grid dimensions (width and height) based on the target resolution defined in the configuration file.

3. Payload Optimization through Tiling Strategy

Due to server-side constraints on the maximum payload size per request, the engine implements a Spatial Tessellation strategy. Large study areas are automatically decomposed into a discrete grid of tiles. The algorithm calculates the number of horizontal and vertical partitions (i, j) and iterates through them, generating specific WCS subsets for each sub-region.

This iterative approach ensures:

1. **Resilience**: It prevents the IGN server from rejecting requests due to excessive data volume.

2. **Scalability**: It allows the processing of large-scale geographic regions that would otherwise exceed memory limits.

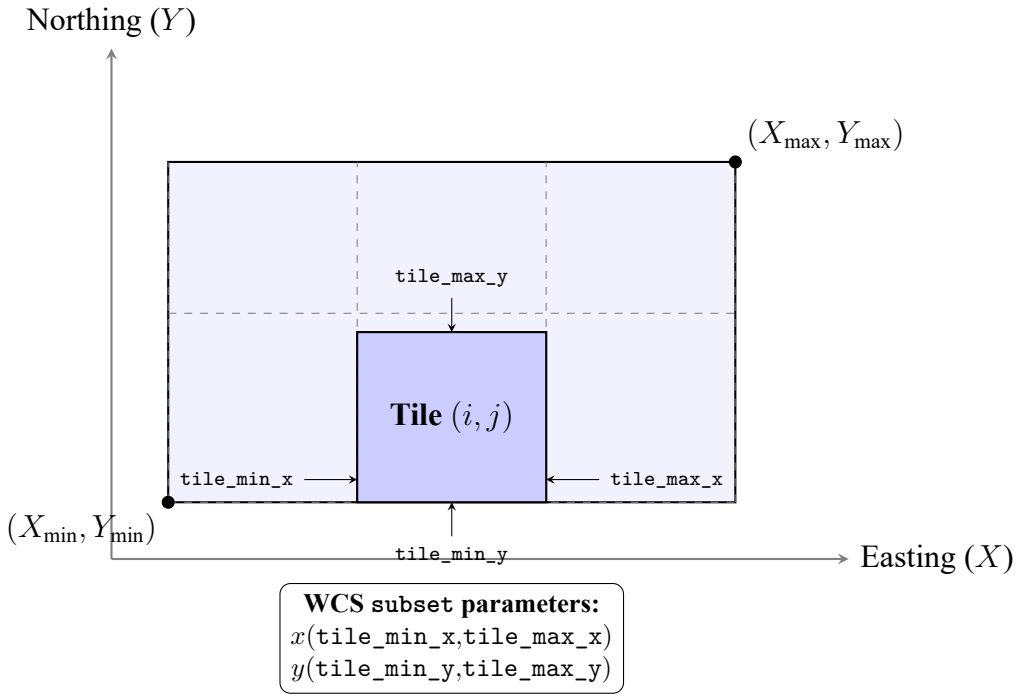

Figure 4.3: Visual representation of the WCS spatial tessellation strategy. The global bounding box of the study area is decomposed into a discrete grid. For each iteration, the engine requests a specific sub-region (Tile i, j) defined by its local coordinates, circumventing server-side payload limitations.

4. Merging and Standardized Persistence

Once all individual tiles are downloaded as in-memory byte streams, the engine delegates the reconstruction to the IdrisiIO handler. The tiles are merged into a continuous mosaic using rasterio, ensuring that overlapping edges and spatial metadata are correctly handled.

Finally, the resulting data is persisted using the standard IDRISI format (.doc and .img), where the elevation values are stored with floating-point precision to maintain the topographic integrity of the simulation environment.

4.2.3.2 Roughness (Manning's n) Engine

This engine transforms land-use classifications into hydraulic roughness coefficients. It maps categorical data e.g., "urban area," "forest," "riverbed") into numerical values used by the transition functions of the Cellular Automata.

- **Type**: Static Layer.
- **Prerequisites**: Requires the Elevation layer to define the study extent.
- **Implementation Details**: [Placeholder: Details on the lookup table (LUT) logic and land-use data sources will be included upon script availability].

4.2.3.3 Water Depth (Initial Conditions) Engine

The Water Depth engine defines the initial hydraulic state of the system. It is responsible for setting the starting water levels in riverbeds or floodplains before the first time step of the simulation.

- **Static (Initial State).**:
- **Defines the $t = 0$ state of the global tensor.**:
- **Implementation Details**: [Placeholder: Details on how initial water levels are derived or loaded from previous simulation states will be included upon script availability].

4.2.3.4 Rainfall Intensity Engine

The Rainfall Intensity Engine is a critical component of the data pipeline, responsible for transforming discrete, asynchronous meteorological observations into a continuous spatiotemporal 3D tensor ($Time \times Y \times X$). This engine ensures that the flood simulator receives high-resolution precipitation forcing data, accounting for both the temporal evolution of storms and the geographic variations influenced by terrain.

The engine is architected following a modular approach, separating the concerns of data acquisition, spatial interpolation, and temporal management.

1. System Architecture and Orchestration

The orchestration of the rainfall data flow is managed by the RainfallGenerator class. This component acts as the high-level controller that integrates three distinct sub-systems:

1. **Extraction Layer (RainDataManager)**: Coordinates multiple data sources (e.g., SAIH Júcar) to fetch raw precipitation records.

2. **Interpolation Layer (KrigingInterpolator)**: Applies advanced geostatistical methods to generalize point data into a continuous surface.

3. **Integration Layer**: Conforms the resulting grids into the simulation's global grid structure, ensuring CRS (Coordinate Reference System) consistency and temporal alignment.

2. Multi-Source Data Acquisition

To ensure reliability and coverage, the engine employs a facade pattern through the RainDataManager. This allows the system to aggregate data from various meteorological networks simultaneously.

- **Standardized Fetching Interface**: All data sources implement a BaseFetcher interface, which enforces methods for station discovery, spatial filtering (within the simulation bounding box), and reprojection of coordinates to the target EPSG.

- **SAIH Júcar Integration**: A specialized fetcher (SAIHJucarFetcher) is implemented to scrape historical data from the "Confederación Hidrográfica del Júcar". This involves a complex process of parsing HTML for station metadata and extracting precipitation values from PDF reports using specialized parsing libraries.

- **Data Normalization**: Raw measurements, often provided in heterogeneous formats or accumulation periods, are normalized into RainDataPoint objects, representing intensity in mm/h at specific spatial coordinates.

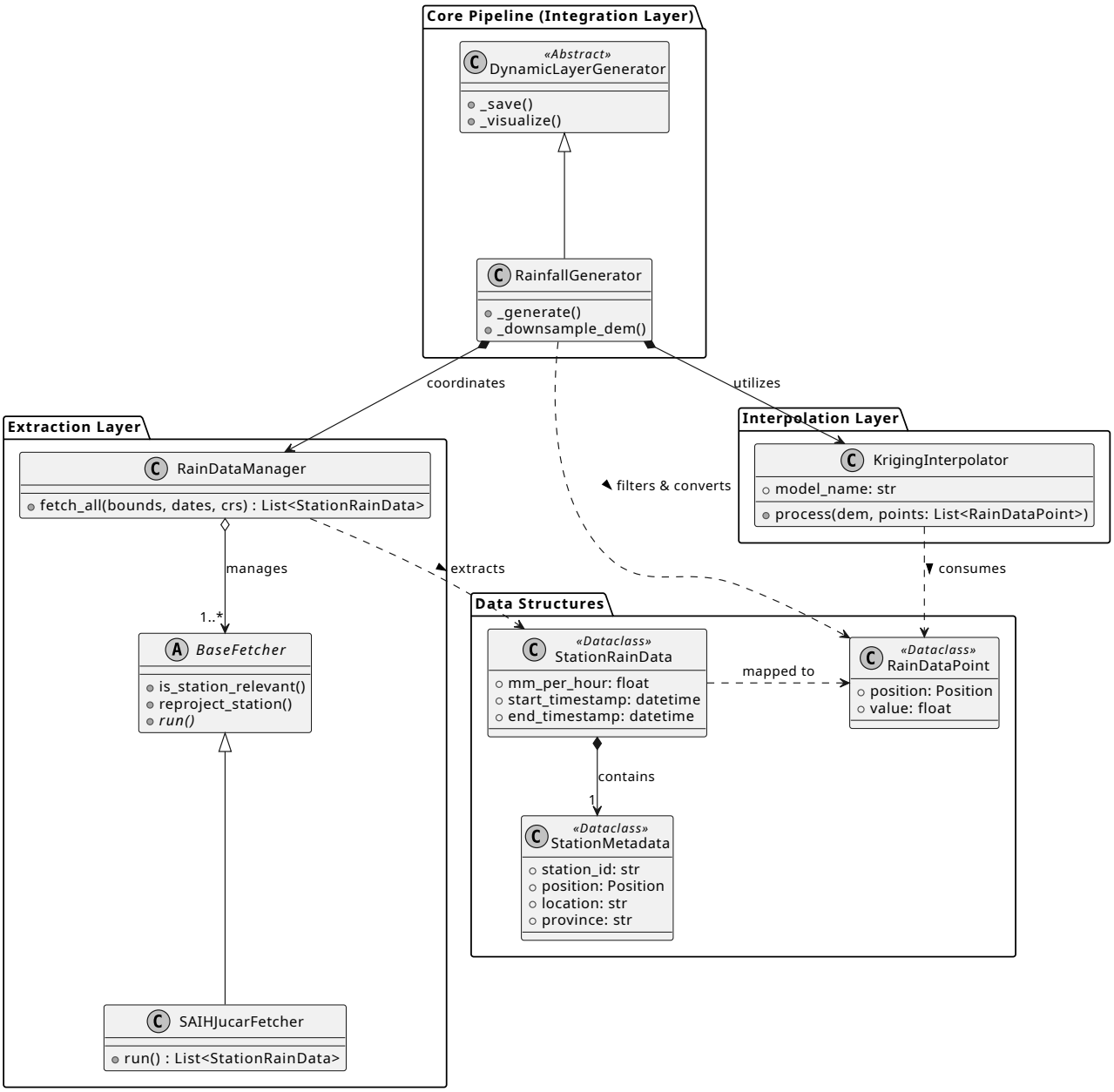

Figure 4.4: UML Class Diagram of the `RainfallGenerator` system architecture. The diagram depicts the structural composition of the module, highlighting the Separation of Concerns (SoC) across the Extraction, Interpolation, and Integration layers. Directional dashed arrows illustrate the flow of specific data structures (`StationRainData`, `RainDataPoint`) as they are fetched from external sources and processed into a unified spatial grid.

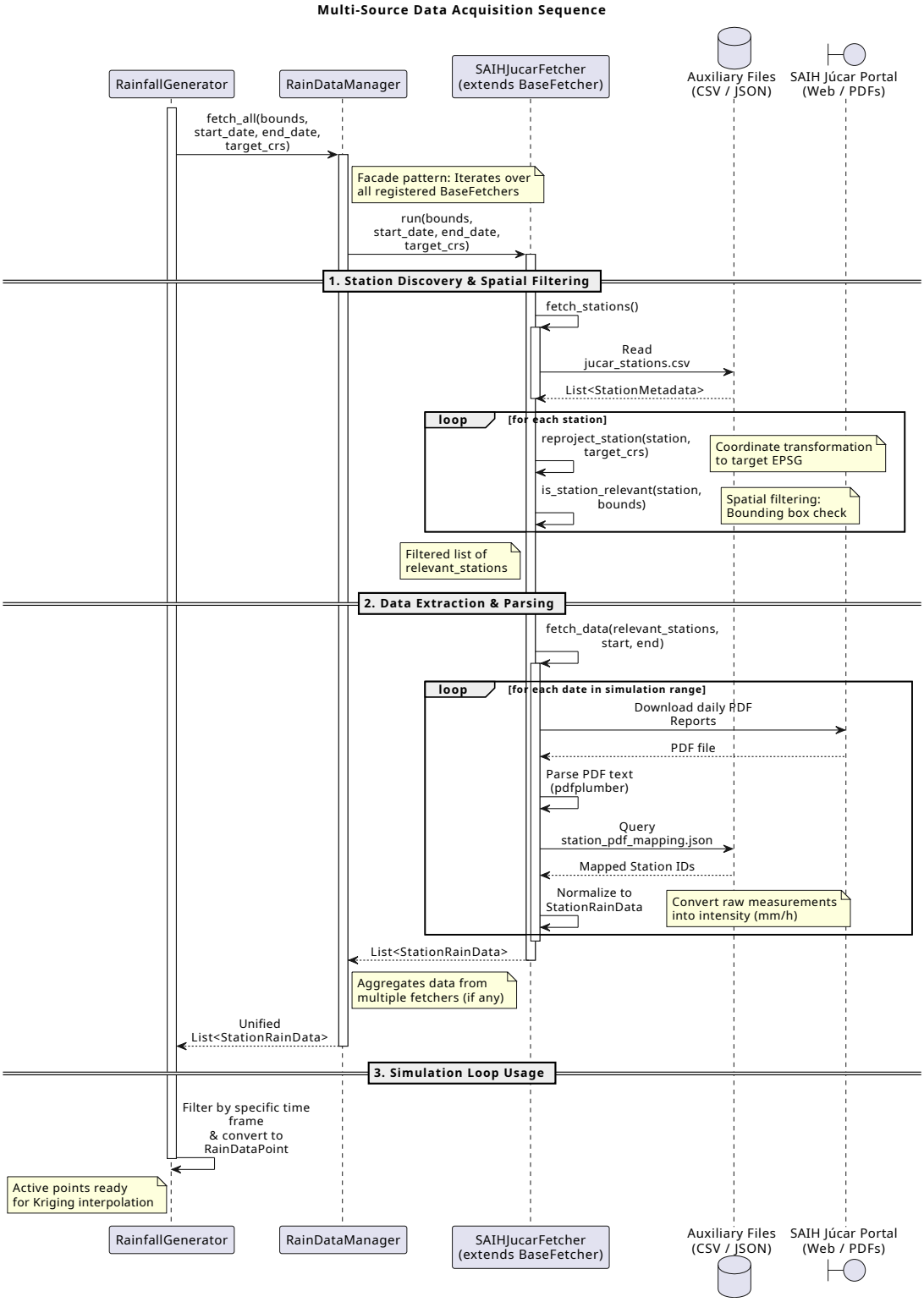

Figure 4.5: The process highlights the role of the `SAIHJucarFetcher` in station discovery, spatial bounding box filtering, coordinate reprojection, and the subsequent extraction and normalization of precipitation data from PDF reports.

3. Spatial Interpolation: Kriging with External Drift (KED)

The engine implements Kriging with External Drift (KED) via the KrigingInterpolator class. Standard interpolation methods (like IDW or Nearest Neighbor) often fail to capture the orographic effect—the phenomenon where higher elevations typically experience higher precipitation intensities. By using a Digital Elevation Model (DEM) as a secondary variable (drift), the engine produces meteorologically consistent surfaces.

The interpolation process follows a rigorous five-step geostatistical workflow:

- **Drift Estimation**: The system extracts elevation values from the DEM at each meteorological station's coordinates. It then fits a linear regression model to establish the relationship:

$$P_{trend} = \alpha + \beta \cdot Z$$

  where P_{trend} is the predicted precipitation and Z is the elevation.

- **Residual Calculation**: To isolate the spatial autocorrelation, the engine subtracts the elevation trend from the observed values at the stations:

$$R(s_i) = P(s_i) - (\alpha + \beta \cdot Z(s_i))$$

- **Variogram Modeling**: Using the gstools library, the engine calculates the experimental variogram of the residuals. It automatically fits a theoretical model (e.g., Spherical, Gaussian, or Exponential) to define the spatial structure, determining parameters such as the nugget, sill, and range.

- **Ordinary Kriging of Residuals**: The engine performs Ordinary Kriging on the residuals across the entire grid. This ensures that local deviations from the elevation trend are smoothly honors at the station locations.

- **Final Reconstruction**: The linear trend is reapplied to the kriged residual surface for every pixel in the grid:

$$P_{final}(x, y) = R_{krige}(x, y) + (\alpha + \beta \cdot Z(x, y))$$

4. Temporal Management and Tensor Assembly

The RainfallGenerator transforms the sequence of interpolated 2D grids into a structured 3D temporal tensor.

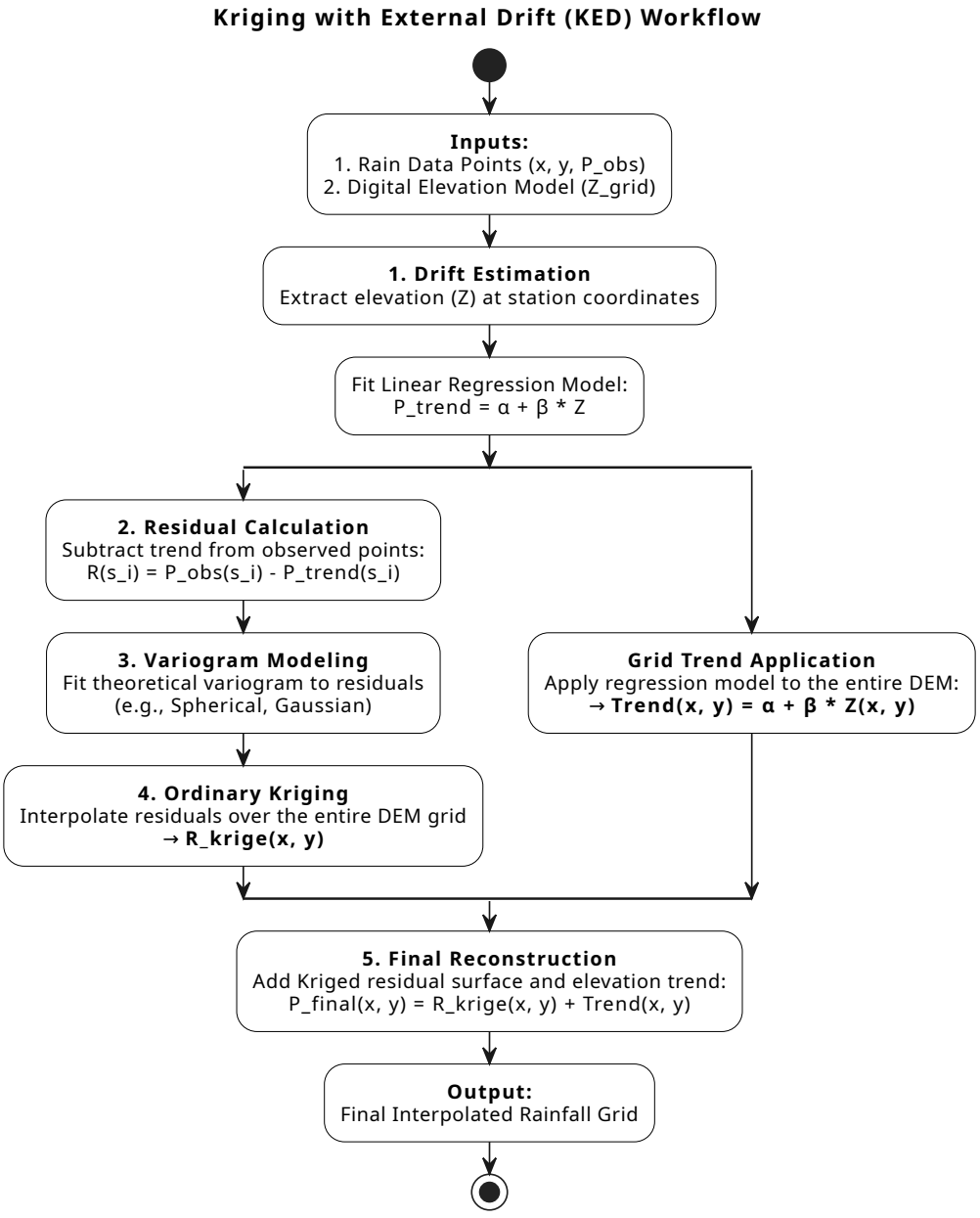

Figure 4.6: Flowchart of the Kriging with External Drift (KED) algorithm. The diagram illustrates the branching process where the global orographic trend is calculated across the entire DEM grid concurrently with the geostatistical interpolation (Ordinary Kriging) of the local residuals. Both branches converge in the final reconstruction step to produce the meteorologically consistent rainfall surface.

- **Temporal Resampling**: Since station observations and simulation time steps may not align, the engine performs temporal filtering. For each simulation frame, it identifies ”active” data points whose measurement windows cover the current timestamp.

- **Resolution Downsampling**: To optimize performance, the engine includes a _downsample_dem utility. This allows the Kriging process to run on a coarser version of the DEM if requested, significantly reducing the computational cost of the variogram fitting without losing the general topographic influence.

- **Data Integrity**: The engine handles edge cases, such as periods with insufficient active stations, by defaulting to zero-rainfall grids or logging warnings to prevent the simulation from receiving uninitialized memory.

4.2.4 Extensibility: Adding New Layers

The implementation’s modularity is best demonstrated by its extensibility. Adding a new physical variable (such as Soil Infiltration or Evapotranspiration) follows a strictly defined developer workflow, as dictated by the architecture in pipeline.py and base.py.

To integrate a new layer, the researcher must:

1. **Define a New Class**: Create a class inheriting from StaticLayerGenerator or DynamicLayerGenerator.

2. **Implement Logic**: Override the _generate method to define the physical transformation.

3. **Register the Component**: Add the class to the DataPipeline registry.

This workflow ensures that the new layer automatically inherits the ability to handle IDRISI/HIF I/O, error logging via Loguru, and automated visualization without writing a single line of extra infrastructure code.

4.2.5 Execution Results and Logging

The observability of the data pipeline is implemented as a multi-sink system that separates high-level operational status from low-level debugging information. This ensures that while the user receives clear progress updates, the full technical history is preserved for auditing.

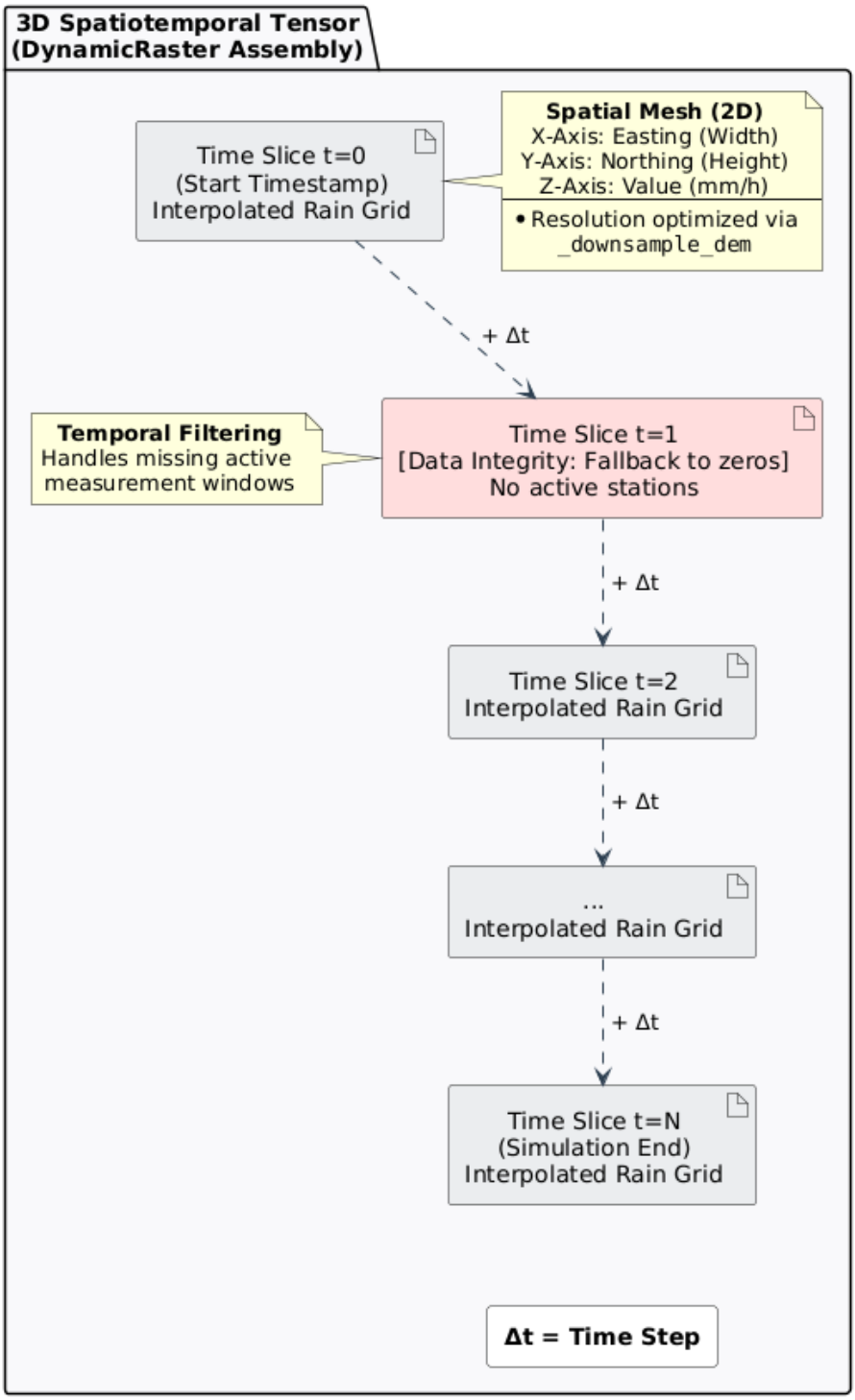

Figure 4.7: Conceptual representation of the 3D spatiotemporal data cube (`DynamicRaster`). Each 2D spatial mesh represents an interpolated rainfall grid at a specific time step (Δt). The chronological stacking illustrates the temporal filtering process, which maintains data integrity by inserting zero-rainfall grids when no active measurement windows are available. Spatial resolution can be optionally optimized via the `_downsample_dem` utility prior to tensor assembly.

4.2.5.1 Dual-Sink Configuration

The logging system is initialized in the main.py entry point. It utilizes a filtered approach to manage the volume of information generated during the processing of large rasters:

- **Standard Output (Console)**: Configured with an INFO level threshold. It uses color-coded tags to highlight milestones (e.g., <green>SUCCESS</green> for completed layers). This sink provides a clean interface, showing only the pipeline initialization, the resolved execution order, and the final completion status.

- **Persistent File (log.txt)**: Configured with a TRACE level threshold. This file captures every granular detail, including the exact file paths being read, the affine transformation matrices extracted by Rasterio, and the specific memory addresses of the processed tensors.

4.2.5.2 Runtime Traceability and Error Handling

The implementation ensures that any failure is not only caught but contextually enriched. As seen in the DataPipeline run loop:

1. **Contextual Breadcrumbs**: Each log entry includes the module name and line number (name:line), allowing developers to pinpoint exactly where a spatial misalignment or I/O error occurred.

2. **Exception Backtracing**: Through the logger.exception() method in the main try-except block, the system automatically captures and formats the full stack trace. This is crucial for debugging complex C++-linked errors that might otherwise be lost in a standard Python traceback.

4.2.5.3 Output Directory Structure

Upon successful execution, the pipeline generates a standardized directory tree. This organization is critical for the C++ simulation core, which expects files in specific locations:

4.2.5.4 Performance and Rotation

To prevent the logging system from becoming a bottleneck during high-frequency data generation, the file sink is implemented with asynchronous writing and rotation policies. The log files are rotated when they reach 10 MB and are automatically compressed into ZIP format, ensuring that the project's storage remains manageable even after hundreds of simulation runs.

```
output_simulation_YYYYMMDD/
├── logs/
│   └── log.txt                          # Full execution
├── elevation/
│   ├── elevation.doc                    # Base terrain (IDRISI)
│   ├── elevation.img
│   └── visualization/
│       └── elevation.png
├── roughness/
│   ├── roughness.doc                    # Manning's n (IDRISI)
│   ├── roughness.img
│   └── visualization/
│       └── roughness.png
├── rainfall/
│   ├── rainfall.hif                     # 3D Rainfall cube (HIF)
│   ├── attrs.json
│   ├── 000000.img
│   ├── 000001.img
│   ├── ...
│   └── visualization/
│       ├── rainfall_frame_001.png       # Temporal snapshots
│       └── ...
└── water_depth/
    ├── water_depth.doc                  # Initial hydraulic state (IDRISI)
    ├── water_depth.img
    └── visualization/
        └── water_depth.png
```

Figure 4.8: Standardized hierarchical directory structure generated by the data pipeline.

4.3 Implementation of the Simulator Core (C++)

4.3.1 Implementation of Hexagonal Architecture

4.3.1.1 Input

Multi-Format Raster File Input Adapter

4.3.1.2 Output

4.3.1.3 State Updater

4.3.2 Algorithm for updating states using tensor operations

4.3.3 Performance optimization and parallelism

4.3.4 Tensor-based Grid Structure

4.3.5 The TemporalLayerManager: Efficient Memory Handling for Dynamic Data

4.4 Integration and Validation

Chapter 5

Conclusions and Future Work

5.1 Conclusions

5.2 Future Work

5.2.1 Integration of Meteorological Radar Data for Enhanced Rainfall Spatialization

While the current data pipeline successfully spatializes point-based rain gauge measurements using elevation-based drift (e.g., Kriging with External Drift), this methodology presents inherent limitations when modeling extreme weather events. Elevation acts as a static covariate that effectively represents long-term, climatological precipitation trends (the orographic effect). However, it cannot capture the highly dynamic, localized, and heterogeneous nature of real-time convective storm cells. Consequently, the interpolated rainfall tensor may misrepresent the exact geographical distribution of the precipitation nucleus during a specific simulation step.

To achieve professional-grade meteorological realism and improve the predictive accuracy of the hydraulic simulation, a primary line of future work is the integration of Meteorological Radar Data (such as the C-band and S-band radar networks operated by the Spanish Meteorological Agency, AEMET).

This integration would fundamentally upgrade the Rainfall Engine through the following technical implementations:

- **Radar Reflectivity Conversion**: Radar systems provide high-resolution spatial matrices of atmospheric reflectivity (Z). The pipeline would need to implement algorithms to dynamically fetch these grids and convert them into estimated rainfall intensity rates (R) using empirical formulas, such as the Marshall-Palmer relationship ($Z = aR^b$).

- **Radar-Rain Gauge Merging**: Because radar data is susceptible to systemic anomalies (e.g., beam blockage, signal attenuation, or overestimation due to hail), it cannot be used in isolation. The future pipeline should implement advanced merging techniques—such as Conditional

Merging (CM) or Regression Kriging. In this upgraded architecture, the radar imagery would serve as the highly detailed spatial covariate, while the physical rain gauges would provide the ”ground-truth” anchor points to correct the radar bias.

- **Enhanced Tensor Fidelity**: By replacing a static topographic covariate with a dynamic meteorological one, the resulting $Time \times Y \times X$ data cube would precisely reflect the trajectory, shape, and intensity of storm events.

Implementing this feature would drastically reduce spatial interpolation errors and provide the Cellular Automata ($m : n - CA^k$) core with a highly realistic input tensor, which is strictly necessary for the accurate modeling of rapid-onset flash floods.

Bibliography

Aleksyuk, A. I., & Belikov, V. V. (2017). Simulation of shallow water flows with shoaling areas and bottom discontinuities. *Computational Mathematics and Mathematical Physics*, *57*, 318–339. https://doi.org/10.1134/S0965542517020026

Beltran, F. S., Boira, V., Vallés-Morán, F. J., & Viadel, R. C. (2025). *Revista fundada per joan fuster*. www.uv.es/lespill

Cockburn, A. (2005). Hexagonal architecture (ports & adapters). https://alistair.cockburn.us/hexagonal-architecture

Dionísio, R., Torres, P. M. B., Ko, K. I., & Lee, M. H. (2025). Mqtt-based architecture for real-time data collection and anomaly detection in smart livestock housing. *Sensors 2025, Vol. 25, Page 7186*, *25*, 7186. https://doi.org/10.3390/S25237186

Fonseca, P., Colls, M., & Casanovas, J. (2011). A novel model to predict a slab avalanche configuration using m:n-cak cellular automata. *Computers, Environment and Urban Systems*, *35*, 12–24. https://doi.org/10.1016/j.compenvurbsys.2010.07.002

i Casas, P. F., & i Palomes, X. P. (2026). Building society 5.0: A foundation for decision-making based on open models and digital twins. *Advanced Engineering Informatics*, *69*. https://doi.org/10.1016/j.aei.2025.103970

Jamali, B., Bach, P. M., Cunningham, L., & Deletic, A. (2019). A cellular automata fast flood evaluation (ca-ffé) model. *Water Resources Research*, *55*, 4936–4953. https://doi.org/10.1029/2018WR023679

Mazzetto, S. (2024). A review of urban digital twins integration, challenges, and future directions in smart city development. *Sustainability 2024, Vol. 16, Page 8337*, *16*, 8337. https://doi.org/10.3390/su16198337

Muñoz, R., Molner, J. V., Campillo-Tamarit, N., & Soria, J. (2025). Estimating peak flows in streams during the flash flood event of 29 october 2024 in spain: An empirical approach. *Water (Switzerland)*, *17*. https://doi.org/10.3390/w17213177

Nunkesser, R. (2022). Using hexagonal architecture for mobile applications. https://doi.org/10.5220/0011075100003266

Vasil'eva, E. S., Belyakova, P. A., Aleksyuk, A. I., Selezneva, N. V., & Belikov, V. V. (2021). Simulating flash floods in small rivers of the northern caucasus with the use of data of automated

hydrometeorological network. *Water Resources*, *48*, 182–193. https://doi.org/10.1134/S0097807821020160